

OS - Chapter 1

فصل 1: مقدمه‌ای بر سیستم‌های عامل

What Operating Systems Do

سیستم‌عامل واسطه بین کاربر و سخت‌افزار است و منابع سیستم (پردازنده، حافظه، I/O) را مدیریت می‌کند.

Computer-System Organization

ساختار سخت‌افزاری شامل CPU، حافظه، دستگاه‌های I/O و ارتباط آن‌ها از طریق باس است.

Computer-System Architecture

بررسی انواع معماری سیستم مانند تک‌پردازنده‌ای، چندپردازنده‌ای (SMP) و خوشه‌ای (Clustered).

Operating-System Operations

عملکرد سیستم‌عامل مبتنی بر وقفه‌ها و حالت‌های اجرایی (کاربر/هسته) است.

Resource Management

مدیریت بهینه‌ی منابع سیستم شامل: CPU، حافظه، دیسک، و I/O توسط سیستم‌عامل.

Security and Protection

محافظت از داده‌ها و جلوگیری از دسترسی غیرمجاز با استفاده از سیاست‌های دسترسی و احراز هویت.

Virtualization

ایجاد نسخه‌های مجازی از منابع (مثل ماشین مجازی) برای اجرای چند سیستم‌عامل روی یک سخت‌افزار.

Distributed Systems

چند سیستم مستقل که از طریق شبکه با هم کار می‌کنند و به اشتراک منابع می‌پردازند.



Kernel Data Structures

استفاده از ساختارهای داده مانند صف، لیست و درخت در هسته برای مدیریت منابع و فرآیندها.



Computing Environments

انواع محیط‌های اجرای سیستم‌عامل: دسکتاپ، موبایل، سیستم‌بلادرنگ، ابری و تعبیه‌شده.



Free/Libre and Open-Source Operating Systems

سیستم‌عامل‌های رایگان و متن‌باز مثل Linux که کاربران می‌توانند آن‌ها را مشاهده و اصلاح کنند.

نکات کلیدی

- سیستم‌عامل، مدیر منابع و رابط بین کاربر و سخت‌افزار است.
- مفاهیم مجازی‌سازی، امنیت، و سیستم‌های توزیع‌شده پایه‌های سیستم‌عامل‌های مدرن هستند.
- هسته از ساختارهای داده برای مدیریت داخلی استفاده می‌کند.



What is an Operating System?

تعریف سیستم‌عامل:

سیستم‌عامل یک **برنامه‌ی سیستمی** است که به عنوان **واسط بین کاربر و سخت‌افزار** عمل می‌کند. کاربر مستقیماً با سخت‌افزار تعامل ندارد؛ بلکه از طریق سیستم‌عامل، برنامه‌های خود را اجرا می‌کند.

اهداف سیستم‌عامل:

- اجرای برنامه‌های کاربر**
→ فراهم کردن بستری برای اجرای نرم‌افزارها و برنامه‌ها.
- راحت کردن حل مسائل برای کاربر**
→ انتزاع پیچیدگی‌های سخت‌افزار (مثلاً استفاده از فایل‌ها به جای کار با سکتور دیسک).
- استفاده‌ی بهینه از سخت‌افزار**
→ تخصیص منابع به صورت کارآمد برای حداکثر بهره‌وری (مثلاً زمان‌بندی CPU).

مثال ساده:

وقتی یک فایل Word را باز می‌کنید:

- شما با Word کار می‌کنید (برنامه‌ی کاربر)،
- سیستم‌عامل به آن دسترسی به دیسک، حافظه، صفحه‌کلید و صفحه‌نمایش می‌دهد،
- کاربر نیازی به دانستن جزئیات سخت‌افزار ندارد.

نکات کلیدی:

- سیستم عامل واسط کاربر و سخت افزار است.
- هدف اصلی: اجرای برنامه ها، سهولت استفاده و استفاده ی بهینه از منابع.
- بدون سیستم عامل، کار با سخت افزار پیچیده و غیر عملی می شود.

Computer System Structure

♦ ساختار یک سیستم کامپیوتری شامل ۴ بخش اصلی است:

1. Hardware (سخت افزار)

منابع فیزیکی اصلی سیستم:

- پردازنده (CPU)
- حافظه (RAM)
- دستگاه های ورودی/خروجی (مثل کیبورد، مانیتور، پرینتر)

2. Operating System (سیستم عامل)

مدیر کل سیستم که:

- سخت افزار را کنترل می کند
- بین برنامه ها و کاربران، منابع را هماهنگ و تخصیص می دهد

3. Application Programs (برنامه های کاربردی)

نرم افزارهایی که با استفاده از منابع سیستم، نیازهای کاربر را رفع می کنند:

- پردازش متن (Word)
- کامپایلرها
- مرورگرها
- پایگاه داده ها
- بازی ها

4. Users (کاربران)

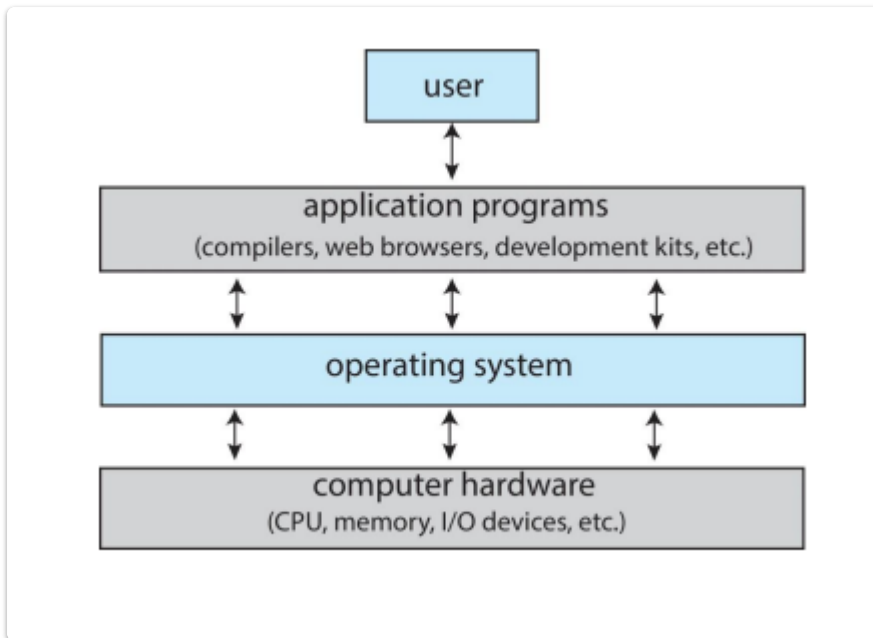
می توانند باشند:

- انسان ها (کاربران نهایی)
- ماشین ها (مثلاً حسگرها)
- یا سایر کامپیوترها (در شبکه)

نکات کلیدی: 📌

- سیستم عامل پلی است بین سخت افزار و نرم افزار کاربردی.
- هر بخش از سیستم نقش خاصی در حل مسئله یا اجرای برنامه دارد.
- کاربران از طریق برنامه ها با سیستم ارتباط برقرار می کنند.

🧱 Abstract View of Components of Computer



این دیاگرام نمایی کلی از اجزای اصلی یک سیستم کامپیوتری را به صورت **لایه ای** نشان می دهد. هر لایه، با لایه های مجاور خود در ارتباط است.

♦ لایه ها از پایین به بالا:

1. Computer Hardware (سخت افزار)

شامل منابع فیزیکی مانند:

- پردازنده (CPU)
- حافظه
- دستگاه های ورودی/خروجی

2. Operating System (سیستم عامل)

لایه ای بین سخت افزار و برنامه های کاربردی که:

- استفاده از سخت افزار را **مدیریت و هماهنگ** می کند
- رابطی استاندارد** برای برنامه ها فراهم می سازد

3. Application Programs (برنامه‌های کاربردی)

نرم‌افزارهایی مانند:

- مرورگر وب، کامپایلر، ویرایشگر کد، برنامه‌های گرافیکی که از خدمات سیستم‌عامل برای انجام وظایفشان استفاده می‌کنند.

4. User (کاربر)

استفاده‌کننده‌ی نهایی که از طریق برنامه‌ها با سیستم تعامل دارد.

📌 نکات کلیدی:

- ساختار سیستم کامپیوتر **لایه‌ای** است.
- سیستم‌عامل نقش **واسط اصلی** بین برنامه‌ها و سخت‌افزار را دارد.
- کاربران فقط از طریق **نرم‌افزارهای کاربردی** با سیستم در تعامل هستند.

🧠 What Operating Systems Do

♦ عملکرد سیستم‌عامل وابسته به نوع کاربرد و دیدگاه کاربران است:

👤 از دید کاربر نهایی:

- کاربران دنبال **راحتی، سهولت استفاده و عملکرد خوب** هستند.
- آن‌ها معمولاً توجهی به چگونگی استفاده از منابع ندارند.

💻 در سیستم‌های اشتراکی (مثل Mainframe):

- سیستم‌عامل باید منابع را بین **چند کاربر** هماهنگ کند.
- نقش سیستم‌عامل:
 - **تخصیص‌دهنده منابع** (Resource Allocator)
 - **برنامه کنترل** برای مدیریت اجرای برنامه‌ها و منابع

💻 در سیستم‌های اختصاصی (مثل Workstation):

- کاربر از منابع خودش استفاده می‌کند
- اما ممکن است از منابع **مشترک شبکه‌ای** مانند چاپگر یا سرور هم بهره ببرد.

📱 در دستگاه‌های موبایل:

- منابع محدود هستند → نیاز به بهینه‌سازی مصرف انرژی و منابع
- تمرکز روی رابط کاربری ساده مانند:
 - صفحه لمسی
 - تشخیص صدا

⚙️ در سیستم‌های نهفته (Embedded Systems):

- معمولاً بدون تعامل مستقیم کاربر
- در وسایلی مانند:
 - اتومبیل‌ها
 - لوازم خانگی
 - روترها
- عملکرد خودکار، بدون رابط کاربری پیچیده

📌 نکات کلیدی:

- نقش سیستم‌عامل بسته به نوع سیستم متفاوت است.
- در بعضی سیستم‌ها هدف، ساده‌سازی برای کاربر است و در بعضی، حداکثر بهره‌برداری از منابع.
- سیستم‌های نهفته معمولاً بدون تعامل انسانی مستقیم کار می‌کنند.

📌 Defining Operating Systems

♦ چرا تعریف سیستم‌عامل پیچیده است؟

عبارت **Operating System (سیستم‌عامل)** معنای ثابتی ندارد چون در کاربردها و طراحی‌های بسیار متنوعی استفاده می‌شود.

♦ نقش‌ها و کاربردهای گسترده سیستم‌عامل:

- در وسایلی مثل:
 - تسترها، یخچال‌ها، تلویزیون‌ها، کنسول‌های بازی
 - کشتی‌ها، فضاپیماها، سیستم‌های صنعتی
- ممکن است بسیار ساده یا بسیار پیچیده باشند:
 - گاهی فقط کنترل یک عملکرد پایه‌ای
 - یا مدیریت هم‌زمان چند برنامه و کاربر

♦ پیدایش سیستم‌عامل‌ها:

- در ابتدا، رایانه‌ها **کاربردهای خاص نظامی** داشتند (برنامه ثابت بدون نیاز به مدیریت منابع)
- با **عمومی شدن رایانه‌ها**، نیاز به سیستمی برای:
 - **مدیریت منابع**
 - **کنترل اجرای برنامه‌ها**
- ایجاد شد → و سیستم‌عامل شکل گرفت.

نکات کلیدی:

- سیستم‌عامل یک **مفهوم واحد نیست** و متناسب با کاربردش تعریف می‌شود.
- نقش سیستم‌عامل از یک کنترل ساده تا مدیریت کامل منابع متغیر است.
- تولد OS نتیجه‌ی **نیاز به کنترل بهتر** در رایانه‌های عمومی‌شده است.

Operating System Definition

♦ چرا تعریف مشخصی برای سیستم‌عامل وجود ندارد؟

چون سیستم‌عامل‌ها از نظر **طراحی، امکانات و کاربرد** بسیار متنوع‌اند، یک تعریف واحد و جامع وجود ندارد. اما چند نگاه رایج وجود دارد:

1. نگاه کاربردی (عملیاتی):

- «هر آن‌چه فروشنده همراه سیستم‌عامل ارائه می‌دهد»
- ♦ شامل کرنل، برنامه‌های سیستمی، ابزارهای مدیریتی و...
- ♦ اما این تعریف از یک سیستم‌عامل به سیستم‌عامل دیگر متفاوت است.

2. نگاه فنی:

- «برنامه‌ای که همیشه در حال اجراست = **Kernel (هسته)**»
- ♦ **کرنل، اصلی‌ترین بخش سیستم‌عامل است:**

- مدیریت سخت‌افزار
- زمان‌بندی پردازنده
- مدیریت حافظه و فرآیندها

3. سایر اجزای سیستم‌عامل:

- **System Programs:** همراه → پشتیبانی، ابزارهای پشته‌بانی از هسته نیستند
- **Application Programs:** هستند OS بازی‌ها که خارج از Word، برنامه‌هایی مثل مرورگر

4. Middleware (میان افزار):

در سیستم عامل های امروزی (مثل ویندوز، اندروید، iOS):

- **میان افزار** مجموعه ای از **فریم ورک ها** و ابزارهاست که خدماتی مثل:
 - بانک اطلاعاتی
 - گرافیک
 - چندرسانه ایبرای توسعه دهندگان برنامه فراهم می کند

نکات کلیدی:

- تعریف سیستم عامل **نسبی و وابسته به دیدگاه** است.
- **کرنل تنها بخش همیشگی در حال اجرا** از سیستم عامل است.
- سیستم عامل های مدرن شامل **میان افزار** برای توسعه ی راحت تر برنامه ها هستند.

مرور کلی ساختار سیستم کامپیوتری (Overview of Computer System Structure)

این اسلاید به بررسی ساختار کلی سیستم کامپیوتری و نحوه عملکرد آن می پردازد.

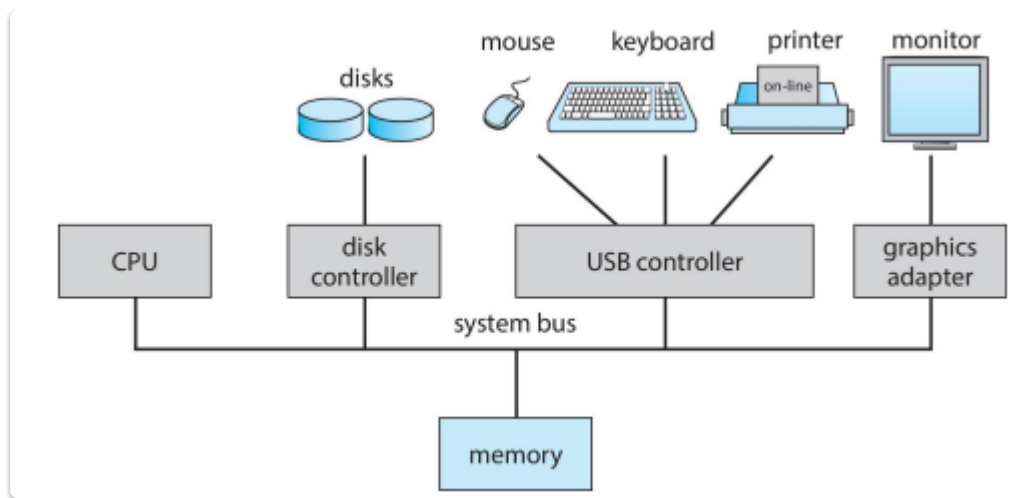
سازمان دهی سیستم کامپیوتری (Computer System Organization)

- **عملکرد سیستم کامپیوتری:**
 - سیستم کامپیوتری معمولاً شامل **یک یا چند CPU** است که وظیفه پردازش داده ها را بر عهده دارند.
 - **کنترل کننده های دستگاه ها (Device Controllers):**
 - این کنترل کننده ها مسئول ارتباط و کنترل دستگاه های جانبی (مانند هارد، صفحه کلید، چاپگر و ...) هستند.
 - همه این بخش ها به وسیله ی یک **Bus مشترک** به هم متصل شده اند که امکان دسترسی به حافظه مشترک را برای CPU ها و دستگاه ها فراهم می کند.

نکته مهم: اجرای هم زمان (Concurrency)

- فعالیت کنند (concurrent) ها و دستگاه ها می توانند به طور **هم زمان** CPU
- آنها برای استفاده از چرخه های حافظه (Memory Cycles) با هم رقابت دارند؛ به این معنی که باید زمان دسترسی به حافظه را مدیریت کنند تا عملکرد سیستم بهینه شود.

نمودار ساختار کلی سیستم کامپیوتری



مثال ساده آموزشی:

تصور کنید شما در یک کتابخانه هستید که چند نفر (CPUها) می‌خواهند به یک کتاب (حافظه) دسترسی پیدا کنند. یک میز (Bus مشترک) بین آنهاست که فقط یکی می‌تواند در هر لحظه کتاب را بردارد و مطالعه کند. پس آنها باید نوبت بگیرند تا بدون برخورد و تداخل از کتاب استفاده کنند.

نکات کلیدی:

- سیستم کامپیوتری شامل یک یا چند CPU و کنترل‌کننده‌های دستگاه است.
- همه این بخش‌ها از طریق یک Bus مشترک به حافظه وصل می‌شوند.
- ها و دستگاه‌ها به صورت هم‌زمان اجرا می‌شوند و برای دسترسی به حافظه رقابت می‌کنند CPU.
- مدیریت این رقابت و دسترسی هم‌زمان، نقش مهمی در کارایی سیستم دارد.

عملکرد سیستم کامپیوتری (Computer-System Operation)

این اسلاید به نحوه تعامل CPU با دستگاه‌های ورودی/خروجی (I/O) و کنترل‌کننده‌های آنها می‌پردازد.

نکات کلیدی درباره عملکرد سیستم:

- اجرای هم‌زمان CPU و دستگاه‌های I/O:** می‌تواند پردازش‌های خود را CPU و دستگاه‌های ورودی/خروجی می‌توانند به طور هم‌زمان فعالیت کنند، یعنی CPU را انجام می‌دهند I/O انجام دهد در حالی که دستگاه‌ها عملیات.
- کنترل‌کننده‌های دستگاه:** هر نوع دستگاه (مثلاً صفحه‌کلید، چاپگر، هارد) یک کنترل‌کننده مخصوص دارد که مسئول مدیریت آن دستگاه است.
- حافظه بافر محلی (Local Buffer):** هر کنترل‌کننده دارای یک حافظه بافر محلی است که داده‌ها را به صورت موقت نگهداری می‌کند. این بافر به عنوان واسطه بین دستگاه و حافظه اصلی عمل می‌کند.
- درایور دستگاه (Device Driver):** برای هر نوع کنترل‌کننده دستگاه، سیستم عامل یک نرم‌افزار به نام درایور دارد که وظیفه مدیریت و کنترل دستگاه را بر

عهده دارد.

- **انتقال داده‌ها:**

داده‌ها را بین حافظه اصلی و بافرهای محلی کنترل‌کننده‌ها جابه‌جا می‌کند CPU.

- **مسیر داده‌ها:**

داده‌های ورودی/خروجی ابتدا از دستگاه به بافر محلی کنترل‌کننده منتقل می‌شوند و سپس CPU آنها را به حافظه اصلی منتقل می‌کند یا برعکس.

- **اطلاع‌رسانی به CPU:**

پس از اتمام عملیات، کنترل‌کننده دستگاه با ایجاد **وقفه (Interrupt)** به CPU اطلاع می‌دهد که کارش تمام شده است.

مثال ساده آموزشی:

فرض کنید شما می‌خواهید یک فایل را از هارد به حافظه انتقال دهید:

- هارد (دستگاه I/O) داده‌ها را به بافر کنترل‌کننده می‌فرستد.
- کنترل‌کننده بعد از اتمام انتقال، با یک وقفه به CPU اطلاع می‌دهد.
- سپس داده‌ها را از بافر محلی به حافظه اصلی منتقل می‌کند CPU.
- در این حین، CPU می‌تواند همزمان کارهای دیگر را انجام دهد بدون اینکه منتظر اتمام عملیات هارد بماند.

نکات کلیدی:

- می‌توانند به طور همزمان فعالیت کنند I/O و دستگاه‌های CPU.
- هر دستگاه یک کنترل‌کننده با بافر محلی دارد.
- درایور سیستم عامل وظیفه مدیریت کنترل‌کننده‌ها را بر عهده دارد.
- داده‌ها بین حافظه اصلی و بافرهای محلی توسط CPU جابه‌جا می‌شوند.
- پس از اتمام عملیات، کنترل‌کننده با ایجاد وقفه، CPU را مطلع می‌کند.

عملکردهای رایج وقفه‌ها (Common Functions of Interrupts)

در این اسلاید به نقش و عملکرد وقفه‌ها (Interrupts) در سیستم عامل پرداخته شده است.

مفهوم وقفه (Interrupt)

- **وقتی وقفه رخ می‌دهد:**
کنترل پردازش به یک برنامه خاص به نام **برنامه سرویس وقفه (Interrupt Service Routine - ISR)** منتقل می‌شود. این انتقال معمولاً از طریق یک ساختار به نام **برداری وقفه (Interrupt Vector)** انجام می‌شود که آدرس همه سرویس‌های وقفه را در خود دارد.
- **ذخیره وضعیت اجرای برنامه:**
هنگام وقوع وقفه، سیستم باید **آدرس دستورالعملی که در حال اجرا بوده را ذخیره کند** تا پس از پایان سرویس وقفه، اجرای برنامه بتواند از همان نقطه ادامه یابد.

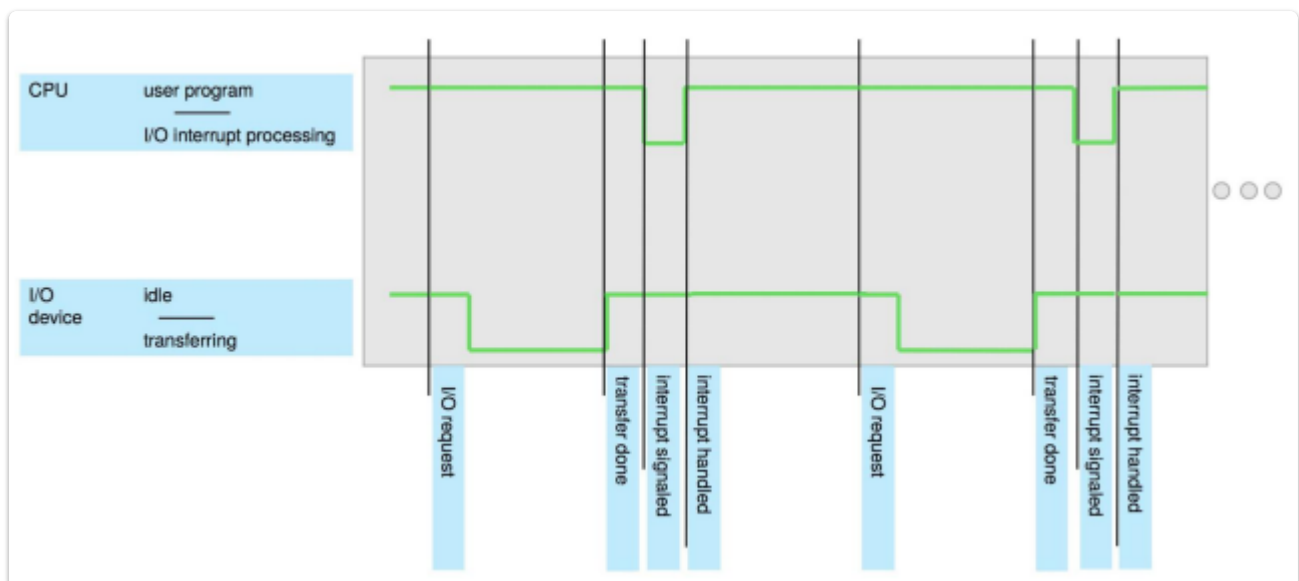
تله (Trap) یا استثناء (Exception)

- تله یا استثناء نوعی وقفه نرم‌افزاری است که توسط خود برنامه یا سیستم به دلیل یک **خطا یا درخواست کاربر** ایجاد می‌شود.
- به عبارتی، برخلاف وقفه‌های سخت‌افزاری که توسط دستگاه‌ها ایجاد می‌شوند، تله‌ها توسط نرم‌افزار تولید می‌شوند.

نقش سیستم عامل در وقفه‌ها

- سیستم عامل به صورت **وقفه‌محور (Interrupt Driven)** طراحی شده است.
- یعنی سیستم عامل منتظر وقوع وقفه‌ها می‌ماند و هرگاه وقفه‌ای رخ دهد، سریعاً آن را مدیریت می‌کند تا پاسخگویی و کارایی سیستم حفظ شود.

نمودار فرآیند وقفه



مثال ساده آموزشی:

فرض کنید برنامه‌ای در حال اجراست و می‌خواهد عدد را بر صفر تقسیم کند، که منجر به یک خطای نرم‌افزاری (Exception) می‌شود:

- این خطا باعث تولید یک **تله (Trap)** می‌شود که به سیستم عامل اطلاع می‌دهد خطایی رخ داده است.
- سیستم عامل با استفاده از برنامه سرویس وقفه مناسب، این خطا را مدیریت کرده و برنامه را کنترل می‌کند.

نکات کلیدی:

- وقفه باعث انتقال کنترل به برنامه سرویس وقفه می‌شود که آدرس آن از طریق بردار وقفه مشخص است.
- آدرس دستور در حال اجرا باید ذخیره شود تا اجرای برنامه بعد از وقفه ادامه یابد.
- تله یا استثناء، وقفه‌های نرم‌افزاری ناشی از خطا یا درخواست کاربر هستند.

- سیستم عامل به صورت وقفه محور عمل می کند و بر اساس وقفه ها تصمیم گیری می کند.

مدیریت وقفه ها (Interrupt Handling)

این اسلاید به نحوه مدیریت وقفه ها توسط سیستم عامل می پردازد.

مراحل مدیریت وقفه توسط سیستم عامل:

1. حفظ وضعیت CPU:

وقتی وقفه رخ می دهد، سیستم عامل ابتدا **وضعیت فعلی CPU** را ذخیره می کند. این شامل ذخیره مقادیر **رجیسترها (Registers)** و **شمارنده برنامه (Program Counter)** است تا اجرای برنامه بتواند بعد از اتمام وقفه به درستی ادامه یابد.

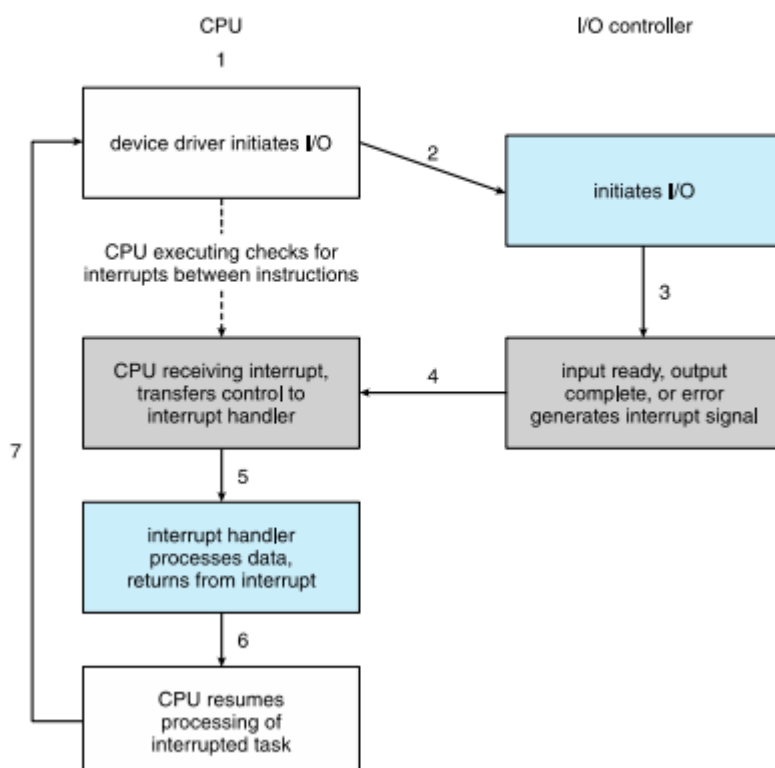
2. تشخیص نوع وقفه:

سیستم عامل باید تشخیص دهد که **چه نوع وقفه ای اتفاق افتاده است** (مثلاً وقفه سخت افزاری، وقفه نرم افزاری، خطا یا درخواست کاربر).

3. اجرای کد مرتبط با وقفه:

برای هر نوع وقفه، **بخش های جداگانه ای از کد (Interrupt Service Routines)** وجود دارد که مشخص می کند سیستم چه عملی باید انجام دهد. سیستم عامل این بخش از کد را اجرا می کند تا وقفه را مدیریت کند.

Interrupt-drive I/O Cycle



مثال ساده آموزشی:

فرض کنید یک وقفه ناشی از پایان انتقال داده از دستگاه ورودی رخ می‌دهد:

- سیستم عامل ابتدا وضعیت CPU را ذخیره می‌کند.
- تشخیص می‌دهد که وقفه مربوط به انتقال داده است.
- سپس ISR مربوط به مدیریت انتقال داده را اجرا می‌کند تا داده‌ها را پردازش کند.
- در نهایت، وضعیت ذخیره شده بازیابی شده و اجرای برنامه اصلی ادامه پیدا می‌کند.

نکات کلیدی:

- سیستم عامل باید وضعیت CPU را به دقت ذخیره کند تا اجرای برنامه پس از وقفه درست ادامه یابد.
- تشخیص نوع وقفه برای انجام عملیات مناسب ضروری است.
- کدهای جداگانه‌ای برای مدیریت انواع مختلف وقفه‌ها وجود دارد.
- پس از مدیریت وقفه، سیستم به اجرای برنامه اصلی باز می‌گردد.

ساختار ورودی/خروجی (I/O Structure)

این اسلاید به دو روش اصلی مدیریت ورودی/خروجی در سیستم‌های عامل اشاره دارد.

دو روش مدیریت ورودی/خروجی:

1. روش اول: انتظار تا پایان عملیات I/O

در این روش، پس از شروع عملیات ورودی/خروجی، کنترل برنامه به کاربر بازگردانده نمی‌شود تا زمانی که عملیات I/O کاملاً به پایان برسد. یعنی برنامه منتظر می‌ماند تا I/O کامل شود و سپس ادامه می‌یابد. این روش سنتی است و در مواقعی که نیاز به هماهنگی دقیق است استفاده می‌شود.

2. روش دوم: بازگشت کنترل بدون انتظار برای اتمام I/O

در این روش، پس از شروع عملیات I/O، کنترل فوراً به برنامه کاربر بازگردانده می‌شود بدون اینکه منتظر پایان عملیات I/O باشد. این امکان می‌دهد که CPU در حین انجام عملیات I/O به کارهای دیگر پردازد و باعث افزایش کارایی سیستم می‌شود. در این روش، سیستم عامل معمولاً از وقفه‌ها (Interrupts) برای اطلاع از پایان I/O استفاده می‌کند.

مثال ساده آموزشی:

فرض کنید می‌خواهید داده‌ای را از دیسک بخوانید:

- روش اول:** برنامه تا زمانی که خواندن کامل نشود متوقف می‌ماند و سپس ادامه می‌یابد.
- روش دوم:** برنامه پس از شروع خواندن به کارهای دیگر پردازد و وقتی خواندن تمام شد، وقفه‌ای دریافت می‌کند و داده‌ها را پردازش می‌کند.

نکات کلیدی:

- دو روش اصلی مدیریت I/O وجود دارد: انتظار برای پایان عملیات و بازگشت فوری کنترل.
- روش انتظار ساده‌تر اما باعث معطل ماندن CPU می‌شود.
- روش بازگشت فوری کنترل باعث بهبود کارایی با استفاده از وقفه‌ها می‌شود.
- سیستم‌های عامل مدرن معمولاً از روش دوم استفاده می‌کنند.

ساختار ورودی/خروجی (ادامه) (I/O Structure - Continued)

در این اسلاید، جزئیات بیشتری درباره دو روش مدیریت I/O و چالش‌های هر روش ارائه شده است.

روش اول: بازگشت کنترل فقط پس از پایان I/O

- **دستور wait:** عملاً بیکار است و منتظر CPU، به پایان برسد. در این حالت I/O منتظر می‌ماند تا عملیات wait با اجرای دستور CPU وقفه بعدی می‌ماند.
- **حلقه انتظار (Wait Loop):** در این روش CPU در حلقه‌ای قرار می‌گیرد که مدام وضعیت I/O را بررسی می‌کند که به آن حلقه انتظار گفته می‌شود. این روش باعث رقابت بر سر دسترسی به حافظه می‌شود و کارایی را کاهش می‌دهد.
- **محدودیت عملیات I/O:** در این روش، حداکثر فقط یک درخواست I/O در هر زمان می‌تواند فعال باشد، بنابراین هیچ عملیات هم‌زمان I/O انجام نمی‌شود.

روش دوم: بازگشت کنترل بدون انتظار برای پایان I/O

- **سیستم کال (System Call):** برنامه کاربر می‌تواند از طریق درخواست سیستم کال به سیستم عامل بگوید که منتظر پایان عملیات I/O بماند یا کارهای دیگر را انجام دهد.
- **جدول وضعیت دستگاه‌ها (Device-Status Table):** سیستم عامل یک جدول دارد که برای هر دستگاه I/O شامل نوع دستگاه، آدرس و وضعیت آن ثبت شده است.
- **مدیریت وقفه‌ها:** سیستم عامل با استفاده از این جدول، وضعیت دستگاه را بررسی کرده و پس از اتمام عملیات I/O، وقفه مربوطه را ثبت و مدیریت می‌کند.

مثال ساده آموزشی:

فرض کنید برنامه‌ای داده‌ای را از هارد می‌خواند:

- در روش اول، CPU منتظر می‌ماند تا خواندن کامل شود و در این مدت نمی‌تواند کار دیگری انجام دهد.
- در روش دوم، CPU پس از شروع خواندن، کارهای دیگر را انجام می‌دهد و وقتی خواندن تمام شد، وقفه‌ای دریافت کرده و داده‌ها را پردازش می‌کند.

نکات کلیدی:

- روش اول باعث بیکار ماندن CPU و رقابت بر سر حافظه می‌شود و فقط یک عملیات I/O هم‌زمان انجام می‌شود.
- روش دوم با استفاده از سیستم کال و جدول وضعیت دستگاه‌ها، مدیریت بهتری برای I/O فراهم می‌کند.
- سیستم‌های عامل مدرن از روش دوم برای بهبود کارایی استفاده می‌کنند.
- جدول وضعیت دستگاه به سیستم عامل اجازه می‌دهد تا به طور دقیق وضعیت هر دستگاه را پیگیری کند.

ساختار حافظه (Storage Structure)

این اسلاید به معرفی ساختارهای حافظه در سیستم کامپیوتری می‌پردازد.

حافظه اصلی (Main Memory)

- تنها رسانه ذخیره‌سازی بزرگی که CPU مستقیماً به آن دسترسی دارد: فقط می‌تواند مستقیماً با حافظه اصلی کار کند و داده‌ها را بخواند یا بنویسد CPU.
- دسترسی تصادفی (Random Access): هر سلول حافظه در زمان تقریباً یکسان و مستقل قابل دسترسی است.
- معمولاً فرار (Volatile): حافظه اصلی معمولاً فرار است؛ یعنی با قطع برق داده‌ها پاک می‌شوند.
- نوع رایج: عمدتاً از حافظه RAM دینامیک (DRAM - Dynamic Random Access Memory) ساخته شده است که هزینه و سرعت مناسبی دارد.

حافظه ثانویه (Secondary Storage)

- افزونه حافظه اصلی: حافظه ثانویه برای ذخیره‌سازی داده‌ها به صورت غیر فرار و با ظرفیت بزرگ استفاده می‌شود.
- غیر فرار (Nonvolatile): داده‌ها حتی پس از خاموش شدن سیستم در این حافظه باقی می‌مانند.
- مثال‌ها: هارد دیسک‌ها، SSDها، دیسک‌های نوری و دیگر رسانه‌های ذخیره‌سازی دائمی.

مثال ساده آموزشی:

فرض کنید می‌خواهید یک برنامه را اجرا کنید:

- برنامه ابتدا از حافظه ثانویه (مثلاً هارد دیسک) به حافظه اصلی (RAM) منتقل می‌شود.
- مستقیماً فقط با حافظه اصلی کار می‌کند و داده‌ها را پردازش می‌کند CPU.
- اگر برق قطع شود، اطلاعات حافظه اصلی پاک می‌شوند اما داده‌ها در حافظه ثانویه باقی می‌مانند.

نکات کلیدی:

- حافظه اصلی، بزرگ‌ترین حافظه‌ای است که CPU مستقیماً به آن دسترسی دارد و معمولاً فرار است.
- حافظه ثانویه ظرفیت بسیار بالاتر و غیر فرار است، ولی CPU مستقیماً به آن دسترسی ندارد.
- داده‌ها و برنامه‌ها برای پردازش ابتدا باید به حافظه اصلی منتقل شوند.

ساختار حافظه (ادامه) (Storage Structure - Continued)

این اسلاید به جزئیات بیشتر درباره حافظه ثانویه، به ویژه دیسک‌های سخت و حافظه‌های غیر فرار جدید می‌پردازد.

هارد دیسک‌ها (Hard Disk Drives - HDD)

- ساختار فیزیکی:** از صفحات سخت فلزی یا شیشه‌ای ساخته شده‌اند که سطح آنها با ماده‌ای مغناطیسی پوشیده شده است.
- ساختار منطقی دیسک:** سطح دیسک به بخش‌هایی به نام **ردیف‌ها (Tracks)** تقسیم شده است، و هر ردیف به بخش‌های کوچکتری به نام **بخش‌ها (Sectors)** تقسیم می‌شود.
- کنترلر دیسک:** یک سخت‌افزار جداگانه به نام کنترلر دیسک، مسئول مدیریت تعامل منطقی بین دیسک و کامپیوتر است، مثلاً درخواست خواندن یا نوشتن داده‌ها.

حافظه‌های غیر فرار (Non-Volatile Memory - NVM)

- ویژگی‌ها:** حافظه‌هایی سریع‌تر از هارد دیسک‌ها هستند و داده‌ها را حتی پس از قطع برق حفظ می‌کنند.
- تکنولوژی‌های مختلف:** انواع مختلفی از این حافظه‌ها وجود دارد، مثل فلش، SSD و غیره.
- روند بازار:** با افزایش ظرفیت و عملکرد و کاهش قیمت، این نوع حافظه‌ها به شدت محبوب‌تر شده‌اند.

مثال ساده آموزشی:

فرض کنید می‌خواهید فایلی را روی هارد دیسک ذخیره کنید:

- سیستم عامل درخواست ذخیره را به کنترلر دیسک می‌فرستد.
- کنترلر محل مناسب در ردیف و بخش دیسک را پیدا کرده و داده‌ها را روی سطح مغناطیسی می‌نویسد.
- حافظه‌های NVM مانند SSD این عملیات را سریع‌تر و بدون قطعات متحرک انجام می‌دهند.

نکات کلیدی:

- هارد دیسک‌ها از صفحات مغناطیسی با ساختار ردیف و بخش تشکیل شده‌اند.
- کنترلر دیسک نقش واسطه بین کامپیوتر و هارد دیسک را دارد.
- حافظه‌های NVM سریع‌تر و غیر فرار هستند و به سرعت جایگزین هارد دیسک‌ها می‌شوند.
- کاهش قیمت و افزایش ظرفیت حافظه‌های NVM باعث محبوبیت بیشتر آنها شده است.

مرور تعاریف و نمادهای حافظه (Storage Definitions and Notation) (Review)

این اسلاید به مفاهیم پایه‌ای درباره اندازه‌گیری و نمایش حافظه در سیستم‌های کامپیوتری می‌پردازد.

واحد پایه ذخیره‌سازی: بیت (Bit)

- **بیت:** کوچک‌ترین واحد داده در کامپیوتر است که می‌تواند یکی از دو مقدار 0 یا 1 را داشته باشد.
- **نمایش داده‌ها:** ترکیب بیت‌ها امکان نمایش انواع داده‌ها را فراهم می‌کند، مانند اعداد، حروف، تصاویر، فیلم‌ها، صداها، اسناد و برنامه‌ها.

واحدهای بزرگ‌تر: بایت، کلمه

- **بایت (Byte):** متشکل از ۸ بیت است و در اکثر کامپیوترها کوچک‌ترین واحد قابل استفاده برای جابجایی داده محسوب می‌شود. برای مثال، بیشتر کامپیوترها دستوراتی برای جابجایی یک بیت ندارند اما برای جابجایی یک بایت دستور دارند.
- **کلمه (Word):** واحد داده بومی معماری کامپیوتر است که از یک یا چند بایت تشکیل شده است. برای مثال، در معماری ۶۴ بیتی، کلمه معمولاً ۸ بایت (۶۴ بیت) است. بسیاری از عملیات کامپیوتر با اندازه کلمه انجام می‌شوند، نه یک بایت به‌تنهایی.

اندازه‌گیری ظرفیت حافظه

- واحدهای حافظه به صورت توانی از ۲ تعریف می‌شوند:
 - **کیلوبایت (KB):** $1,024$ بایت (2^{10})
 - **مگابایت (MB):** $1,024^2$ بایت (2^{20})
 - **گیگابایت (GB):** $1,024^3$ بایت (2^{30})
 - **ترابایت (TB):** $1,024^4$ بایت (2^{40})
 - **پتابایت (PB):** $1,024^5$ بایت (2^{50})
- **گردش تولیدکنندگان:** برخی تولیدکنندگان ظرفیت‌ها را به صورت تقریبی و ده‌دهی (مثلاً ۱ میلیون بایت برای مگابایت) اعلام می‌کنند که با مقدار دقیق کمی تفاوت دارد.
- **استثنا در شبکه‌ها:** در شبکه‌ها اندازه‌گیری‌ها معمولاً بر حسب بیت است، چون انتقال داده‌ها به صورت بیت به بیت انجام می‌شود.

مثال ساده آموزشی:

- فرض کنید یک فایل متنی کوچک دارید:
- این فایل از 10,240 بایت (10 کیلوبایت) تشکیل شده است.
- این مقدار برابر با $81,920 = 8 \times 10,240$ بیت است.
- کامپیوتر داده‌ها را معمولاً به صورت بایت یا کلمه پردازش می‌کند، نه بیت به بیت.

نکات کلیدی:

- بیت کوچک‌ترین واحد داده است؛ بایت از ۸ بیت تشکیل شده.
- کلمه واحد داده معماری کامپیوتر است و معمولاً چند بایت دارد.
- ظرفیت حافظه‌ها معمولاً به صورت توان‌های ۲ (مثلاً 1024) تعریف می‌شود.
- تولیدکنندگان حافظه معمولاً مقادیر ظرفیت را به صورت تقریبی ده‌دهی هم اعلام می‌کنند.
- شبکه‌ها داده‌ها را به صورت بیت جابجا می‌کنند، به همین دلیل اندازه‌گیری‌ها بر حسب بیت است.

سلسله‌مراتب حافظه (Storage Hierarchy)

سلسله‌مراتب حافظه

سیستم‌های ذخیره‌سازی معمولاً به صورت سلسله‌مراتبی سازمان‌دهی می‌شوند که ویژگی‌های زیر را در نظر می‌گیرد:

- سرعت (Speed):** هرچه به بالای سلسله‌مراتب نزدیک‌تر باشیم، سرعت دسترسی سریع‌تر است.
- هزینه (Cost):** هرچه به پایین سلسله‌مراتب برویم، هزینه ذخیره‌سازی به ازای هر بیت کمتر می‌شود.
- فرار بودن (Volatility):** حافظه‌های بالاتر معمولاً فرار (مثل RAM) و حافظه‌های پایین‌تر غیر فرار (مثل هارد دیسک) هستند.

کشینگ (Caching)

- کشینگ یعنی **کپی کردن اطلاعات به یک حافظه سریع‌تر** برای افزایش سرعت دسترسی.
- مثلاً، **حافظه اصلی (RAM)** را می‌توان به عنوان یک کش برای حافظه ثانویه (هارد دیسک) در نظر گرفت.

درایور دستگاه (Device Driver)

- هر کنترلر دستگاه یک **درایور دستگاه** دارد که وظیفه مدیریت ورودی/خروجی (I/O) را بر عهده دارد.
- درایور دستگاه یک **رابط یکنواخت** بین کنترلر سخت‌افزاری و هسته (kernel) سیستم عامل فراهم می‌کند تا مدیریت دستگاه‌ها ساده‌تر شود.

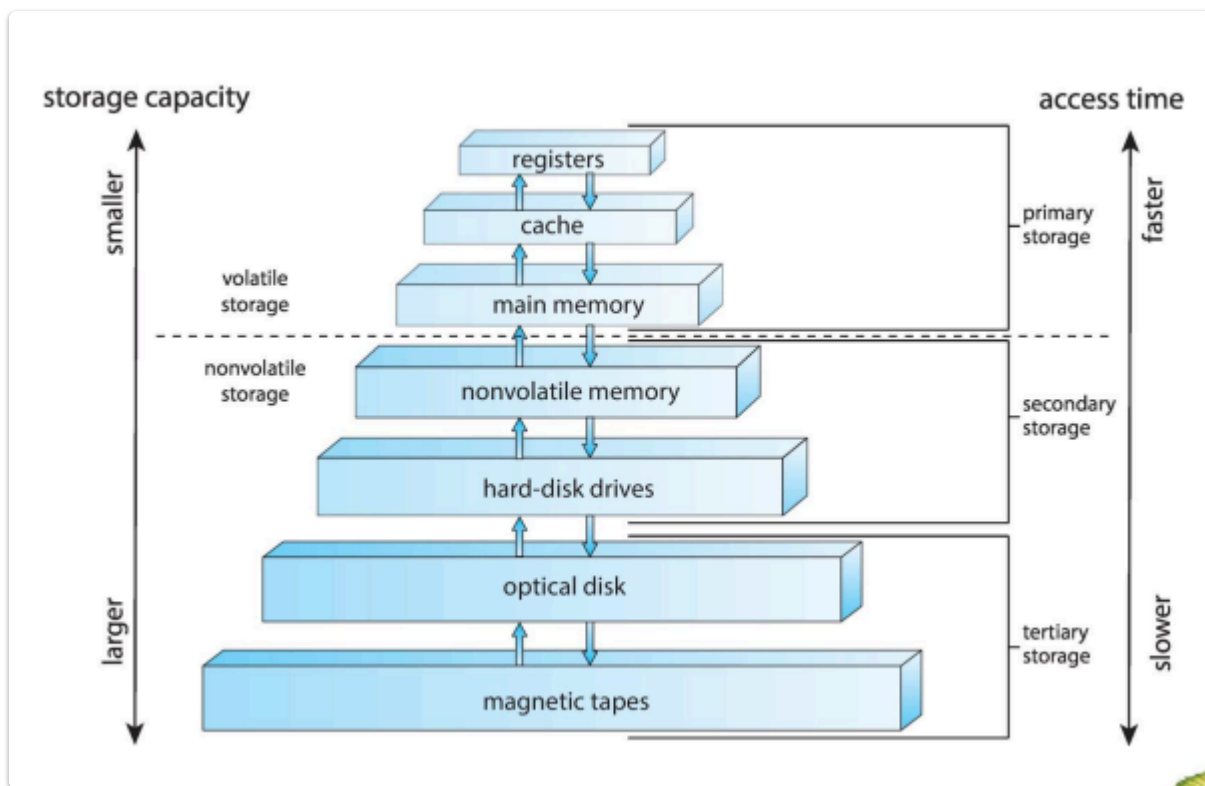
مثال ساده آموزشی:

فرض کنید می‌خواهید یک فایل را باز کنید:

- ابتدا سیستم عامل تلاش می‌کند داده‌ها را از **کش** (اگر موجود باشند) بخواند که سریع است.
- اگر در کش نباشد، به حافظه اصلی مراجعه می‌کند.
- در صورت عدم دسترسی در حافظه اصلی، داده‌ها از حافظه ثانویه (مثلاً هارد دیسک) بارگذاری می‌شوند.
- همه تعامل‌ها با دستگاه‌های سخت‌افزاری توسط درایورهای دستگاه مدیریت می‌شود تا سیستم عامل بتواند به شکل یکنواخت با آنها کار کند.

نکات کلیدی:

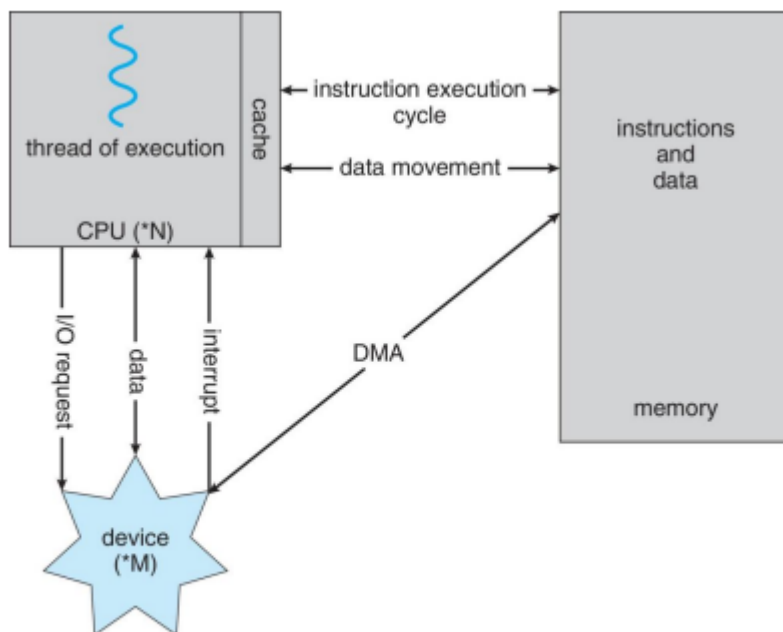
- سیستم‌های ذخیره‌سازی به صورت سلسله‌مراتبی از نظر سرعت، هزینه و فرار بودن سازمان‌دهی می‌شوند.
- کشینگ باعث افزایش سرعت دسترسی با استفاده از حافظه‌های سریع‌تر می‌شود.
- درایور دستگاه رابط بین سخت‌افزار و هسته سیستم عامل است و مدیریت I/O را انجام می‌دهد.



نحوه کار یک کامپیوتر مدرن (How a Modern Computer Works)

این دیاگرام نمایی ساده از **معماری فون نویمان (von Neumann architecture)** را نمایش می‌دهد که در آن **دستورالعمل‌ها و داده‌ها در یک حافظه مشترک** قرار دارند. این مدل پایه‌ای برای اکثر سیستم‌های کامپیوتری امروزی است.

اجزای اصلی نمایش داده شده در دیاگرام:



A von Neumann architecture

1. پردازنده مرکزی (CPU):

- دارای یک یا چند رشته اجرایی (thread of execution) است که دستورالعمل‌ها را اجرا می‌کنند.
- به کمک حافظه کش (cache)، داده‌ها و دستورالعمل‌ها را سریع‌تر دریافت می‌کند.
- با حافظه اصلی برای اجرای دستورالعمل‌ها و جابجایی داده‌ها تعامل دارد.

2. Memory (حافظه):

- شامل دستورالعمل‌ها و داده‌ها است.
- حافظه به صورت مشترک برای داده و برنامه استفاده می‌شود (مشخصه فون نویمان).
- منبعی است که CPU داده‌ها را از آن می‌خواند یا در آن می‌نویسد.

3. دستگاه I/O - Device (*M):

- دستگاه‌های ورودی/خروجی مانند دیسک‌ها، کارت‌های شبکه، صفحه‌کلید و غیره.
- از طریق درخواست I/O با CPU ارتباط برقرار می‌کنند.
- پس از اتمام عملیات، با ایجاد وقفه (Interrupt) به CPU اطلاع می‌دهند.
- داده‌ها را از/به حافظه جابه‌جا می‌کنند.

4. DMA (Direct Memory Access):

- مکانیزمی است که دستگاه‌های I/O می‌توانند بدون دخالت مستقیم CPU، داده‌ها را مستقیماً بین حافظه و خودشان انتقال دهند.
- باعث بهینه‌سازی و آزاد شدن CPU برای انجام کارهای دیگر می‌شود.

چرخه عملیات:

1. CPU (Instruction Execution Cycle) دستورالعمل‌ها را از حافظه می‌گیرد.
2. هنگام نیاز به داده یا نوشتن خروجی، CPU دستور درخواست I/O به دستگاه صادر می‌کند.
3. دستگاه I/O پس از انجام عملیات، از طریق وقفه (Interrupt) به CPU اطلاع می‌دهد.

4. در صورت استفاده از **DMA**، انتقال داده بین حافظه و دستگاه بدون دخالت مستقیم CPU انجام می‌شود.
5. حافظه کش، برای تسریع انتقال داده بین CPU و حافظه استفاده می‌شود.

مثال ساده آموزشی:

فرض کنید شما در حال اجرای برنامه‌ای برای باز کردن یک تصویر از دیسک هستید:

1. دستور «باز کردن فایل» را اجرا می‌کند CPU.
2. یک درخواست I/O به دستگاه ذخیره‌سازی ارسال می‌شود.
3. دستگاه داده‌ها را از دیسک می‌خواند.
4. داده‌ها از طریق DMA مستقیماً به حافظه منتقل می‌شوند.
5. پس از اتمام خواندن، دستگاه به CPU «وقفه» ارسال می‌کند.
6. پس از دریافت وقفه، ادامه اجرای برنامه را انجام می‌دهد CPU.

نکات کلیدی:

- معماری فون نویمان، مدل پایه‌ای برای اکثر کامپیوترهای مدرن است.
- دستورالعمل‌ها و داده‌ها هر دو در حافظه مشترک قرار دارند.
- ارتباط CPU با حافظه و دستگاه‌ها از طریق I/O، وقفه، و گاهی DMA انجام می‌شود.
- استفاده از کش و DMA باعث بهبود کارایی سیستم می‌شود.

ساختار دسترسی مستقیم به حافظه (Direct Memory Access) (Structure - DMA)

این اسلاید به بررسی مکانیزم **دسترسی مستقیم به حافظه (DMA)** می‌پردازد که برای **دستگاه‌های ورودی/خروجی با سرعت بالا** طراحی شده‌است تا کارایی سیستم افزایش یابد.

مفهوم اصلی DMA:

در حالت عادی، هنگام انتقال داده بین یک دستگاه (مثلاً دیسک) و حافظه، CPU باید درگیر تمام مراحل خواندن یا نوشتن باشد. این کار باعث اشغال CPU و کاهش کارایی سیستم می‌شود.

اما با استفاده از DMA:

- کنترلر دستگاه مستقیماً داده‌ها را بین **بافر دستگاه** و **حافظه اصلی** منتقل می‌کند.
- **نیازی به مداخله در انتقال هر بایت ندارد CPU**.
- فقط **یک وقفه (interrupt)** به ازای هر بلاک (نه هر بایت) به CPU ارسال می‌شود.

مزایا:

- کاهش بار روی CPU
- افزایش سرعت انتقال داده
- مناسب برای دستگاههایی با نرخ انتقال بالا مانند دیسک‌ها، کارت‌های شبکه و کارت‌های گرافیک

مثال ساده آموزشی:

فرض کنید قصد دارید یک ویدیو از دیسک سخت پخش کنید:

- به جای اینکه CPU هر فریم را بخواند و در حافظه قرار دهد، این وظیفه به کنترلر دیسک داده می‌شود.
- کنترلر دیسک با استفاده از DMA داده‌های ویدیو را مستقیماً به حافظه اصلی می‌فرستد.
- پس از اتمام ارسال هر بلاک، فقط یک وقفه به CPU ارسال می‌شود تا ادامه فرآیند (مثلاً پخش فریم‌ها) انجام شود.

نکات کلیدی:

- به دستگاه‌های سریع اجازه می‌دهد تا مستقیماً با حافظه تعامل کنند DMA.
- باعث کاهش تعداد وقفه‌ها و آزادسازی CPU برای انجام سایر وظایف می‌شود.
- تنها یک وقفه به ازای هر بلاک داده ارسال می‌شود، نه هر بایت.

عملیات سیستم‌عامل (Operating-System Operations)

این اسلاید مراحل اولیه راه‌اندازی سیستم‌عامل و نحوه مدیریت عملیات توسط هسته (Kernel) را توضیح می‌دهد. همچنین به نقش وقفه‌ها (Interrupts) در تعامل با سخت‌افزار و نرم‌افزار پرداخته می‌شود.

مراحل اولیه راه‌اندازی سیستم:

1. Bootstrap Program:

- قطعه‌ای از کد ساده در حافظه فقط‌خواندنی (ROM) یا فلش است.
- وظیفه‌ی آن **مقدمه‌سازی سخت‌افزار و بارگذاری سیستم‌عامل (Kernel)** به حافظه است.
- به این فرآیند **Booting (راه‌اندازی)** گفته می‌شود.

2. Kernel Loads:

- پس از اجرای Bootstrap، هسته سیستم‌عامل در حافظه بارگذاری می‌شود.
- هسته کنترل کامل سیستم را بر عهده می‌گیرد.

3. اجرای Daemon ها (فرآیندهای پس‌زمینه):

- برنامه‌هایی هستند که خدمات سیستمی فراهم می‌کنند (مثل مدیریت چاپگر، شبکه، ساعت) **Daemon**.
- خارج از هسته اجرا می‌شوند ولی بخشی از کارکرد اصلی سیستم‌عامل‌اند.

وقفه‌ها (Interrupts) در سیستم‌عامل:

سیستم‌عامل بر اساس وقفه‌ها عمل می‌کند تا کارها را مدیریت کند.

1. وقفه سخت‌افزاری (Hardware Interrupt):

- توسط دستگاه‌های I/O ارسال می‌شود (مثلاً اتمام خواندن از دیسک).
- باعث قطع اجرای فعلی CPU برای رسیدگی به نیاز دستگاه می‌شود.

2. وقفه نرم‌افزاری (Software Interrupt یا Trap):

- توسط نرم‌افزار ایجاد می‌شود، به دو دلیل:
- **خطاهای نرم‌افزاری:** مثل تقسیم بر صفر، دسترسی غیرمجاز به حافظه.
- **درخواست خدمات سیستم‌عامل:** مثل `system call` ها برای باز کردن فایل یا تخصیص حافظه.

3. سایر مشکلات ممکن:

- حلقه‌های بی‌نهایت (infinite loop)
- فرآیندهایی که سعی در تغییر داده‌های فرآیندهای دیگر یا خود سیستم‌عامل دارند.

مثال آموزشی ساده:

فرض کنید کامپیوتر خود را روشن می‌کنید:

1. BIOS (bootstrap) اجرا شده و حافظه را بررسی می‌کند.
2. سیستم‌عامل را از دیسک بارگذاری می‌کند.
3. هسته سیستم‌عامل شروع به کار کرده و سرویس‌هایی مثل شبکه یا نمایشگر را فعال می‌کند.
4. اگر در برنامه‌ای تقسیم بر صفر رخ دهد، یک وقفه نرم‌افزاری به هسته ارسال می‌شود تا با آن برخورد کند (مثلاً خاتمه دادن به برنامه).

نکات کلیدی:

- فرآیند Boot با یک کد ساده در ROM شروع می‌شود.
- هسته پس از بارگذاری، کنترل سیستم را به دست می‌گیرد.
- سیستم‌عامل بر اساس وقفه‌ها، هم از سخت‌افزار و هم از نرم‌افزار، عمل می‌کند.
- وقفه‌ها امکان پاسخ‌گویی سریع به رویدادها و مدیریت خطا را فراهم می‌کنند.

سیستم چندبرنامه‌ای (Multiprogramming) - سیستم‌های دسته‌ای / (Batch Systems)

در این اسلاید به مفهوم **چندبرنامه‌ای** پرداخته شده است؛ یک تکنیک کلیدی در طراحی سیستم‌عامل‌ها برای **افزایش بهره‌وری CPU** و جلوگیری از اتلاف منابع.

مشکل سیستم‌های ساده:

در سیستم‌های ابتدایی:

- فقط یک کاربر در هر لحظه می‌توانست برنامه‌اش را اجرا کند.
- وقتی برنامه منتظر ورودی/خروجی (I/O) بود، CPU بیکار می‌ماند.

راه‌حل: چندبرنامه‌ای (Multiprogramming)

مفهوم Multiprogramming:

- چند برنامه (Job) به طور هم‌زمان در حافظه نگه‌داری می‌شوند.
- (نیست I/O یعنی منتظر) برنامه‌ای را اجرا می‌کند که آماده اجرا است OS.
- اگر برنامه در حال اجرا نیاز به I/O داشت، OS بلافاصله آن را تعلیق کرده و به اجرای برنامه‌ای دیگر می‌پردازد.
- این کار از طریق زمان‌بندی کارها (Job Scheduling) انجام می‌شود.

شرایط لازم برای چندبرنامه‌ای:

- سیستم باید دارای مدیریت حافظه برای نگهداری چند برنامه به صورت هم‌زمان باشد.
- زمان‌بندی CPU برای انتخاب برنامه بعدی.
- کنترل مناسب روی ورودی/خروجی برای تشخیص زمان پایان عملیات I/O.

مثال ساده:

فرض کنید سه برنامه در حال اجرا باشند:

- I/O در حال خواندن داده از دیسک است → منتظر Job1
- کار می‌کند CPU سوئیچ می‌کند که با Job2 بلافاصله به OS
- سپس Job2 نیز منتظر چاپگر می‌ماند → حالا نوبت Job3 می‌شود.
- به این ترتیب، CPU همیشه مشغول اجراست.

نکات کلیدی:

- چندبرنامه‌ای باعث افزایش استفاده مؤثر از CPU می‌شود.
- در حافظه فقط زیرمجموعه‌ای از کل برنامه‌ها قرار می‌گیرند.
- زمان‌بندی شغل‌ها برای انتخاب برنامه بعدی استفاده می‌شود.
- پایه‌ی سیستم‌های Batch و پایه اولیه برای چندوظیفگی (Multitasking) مدرن است.

چندوظیفگی (Multitasking - Time Sharing)

این اسلاید به مفهوم **چندوظیفگی زمان‌مشارکتی (Time Sharing)** می‌پردازد که یک گام پیشرفته‌تر از سیستم‌های چندبرنامه‌ای (Multiprogramming) محسوب می‌شود و زمینه‌ساز **محاسبات تعاملی (Interactive Computing)** است.

تفاوت با چندبرنامه‌ای (Multiprogramming):

چندبرنامه‌ای (Batch)	چندوظیفگی (Time Sharing)
تمرکز بر استفاده بهینه از CPU	تمرکز بر تعامل کاربر با سیستم
برای محاسبات غیرتعاملی طراحی شده	برای محاسبات تعاملی و کاربران همزمان طراحی شده
سیستم خودش برنامه‌ها را زمان‌بندی می‌کند	کاربران می‌توانند با برنامه‌های خود تعامل کنند

ویژگی‌های Multitasking (Time Sharing):

- توسعه‌ی منطقی سیستم‌های Batch است.
- با سرعت بالا سوئیچ می‌کند، طوری که کاربران احساس می‌کنند برنامه‌شان در حال اجرا (Jobs) بین برنامه‌ها CPU است.
- هر کاربر حداقل یک برنامه در حال اجرا در حافظه دارد که به آن **Process (فرآیند)** گفته می‌شود.
- اگر چند فرآیند همزمان آماده اجرا باشند، از **زمان‌بندی CPU** برای تخصیص زمان به آن‌ها استفاده می‌شود.

اجرای فرآیندها در حافظه:

- اگر تعداد فرآیندها بیشتر از حافظه موجود باشد:
- سیستم‌عامل از **Swapping** برای جابجایی فرآیندها بین حافظه و دیسک استفاده می‌کند.
- برای اجرای برنامه‌هایی که **کامل در حافظه جا نمی‌شوند**، از **Virtual Memory** استفاده می‌شود:
- یعنی بخش‌هایی از برنامه در حافظه قرار می‌گیرد و بقیه در دیسک باقی می‌ماند.
- سیستم به صورت خودکار آن‌ها را در صورت نیاز جابجا می‌کند.

مثال ساده:

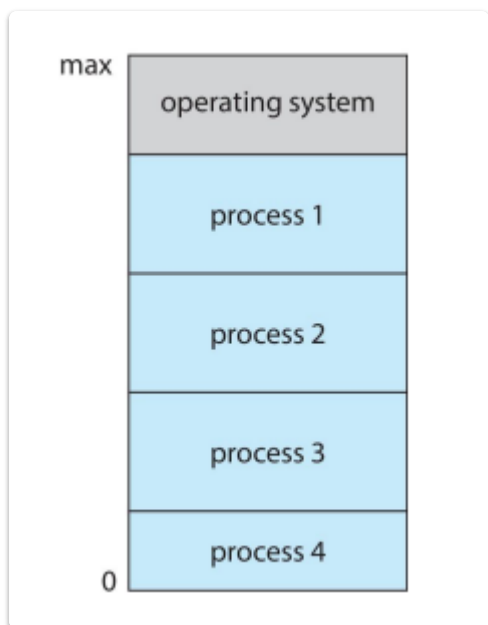
- 3 کاربر هم‌زمان با سیستم کار می‌کنند: یکی تایپ می‌کند، یکی در حال مشاهده ویدیو است، دیگری فایل باز می‌کند.
- به هر فرآیند یک بازه زمانی بسیار کوتاه (مثلاً 10 میلی‌ثانیه) اختصاص می‌دهد CPU.
- این چرخش سریع باعث می‌شود هر کاربر **احساس کند تنها او از سیستم استفاده می‌کند**.

نکات کلیدی:

- Multitasking = Multiprogramming + تعامل کاربران
- پاسخ سیستم باید سریع باشد (معمولاً > 1 ثانیه)
- هر فرآیند باید در حافظه باشد یا توسط **Virtual Memory** پشتیبانی شود

- زمانی که فرآیندی منتظر I/O است، CPU به فرآیند دیگری اختصاص می‌یابد
- سیستم‌عامل باید توانایی **زمان‌بندی دقیق، مدیریت حافظه، و تعامل هم‌زمان با کاربران متعدد** را داشته باشد

Memory Layout for Multiprogrammed System



این تصویر، **چیدمان حافظه (Memory Layout)** را در یک سیستم چندبرنامه‌ای (Multiprogrammed System) نشان می‌دهد. در چنین سیستمی، چند فرآیند به طور هم‌زمان در حافظه بارگذاری می‌شوند تا CPU همیشه برنامه‌ای برای اجرا داشته باشد.

توضیح کامل تصویر:

- در بخش بالایی حافظه قرار دارد (نزدیک به آدرس‌های بالا) **Operating System**.
- در ادامه‌ی حافظه قرار گرفته‌اند **Process 1 تا Process 4**.
- این چیدمان خطی است، یعنی هر فرآیند بخشی از حافظه را اشغال کرده است، به ترتیبی که سیستم‌عامل اجازه داده.

مفاهیم مرتبط:

چرا چندبرنامه‌ای؟

در یک سیستم تک‌وظیفه‌ای، اگر یک برنامه منتظر I/O باشد، CPU بیکار می‌ماند. اما در سیستم چندبرنامه‌ای:

- وقتی یک فرآیند منتظر I/O است، CPU به فرآیند دیگری اختصاص داده می‌شود.
- این امر باعث **افزایش بهره‌وری CPU** و **کاهش زمان بیکاری سیستم** می‌شود.

نحوه مدیریت این حافظه:

برای مدیریت این چیدمان حافظه، سیستم‌عامل باید:

- برای هر فرآیند فضای جداگانه‌ای در حافظه رزرو کند.
- از جدول مدیریت حافظه برای پیگیری موقعیت و اندازه هر فرآیند استفاده کند.
- از وقوع تداخل بین فرآیندها (مثلاً دسترسی به حافظه یکدیگر) جلوگیری کند.

نکات کلیدی:

- در سیستم چندبرنامه‌ای، حافظه بین چند فرآیند به اشتراک گذاشته می‌شود.
- سیستم‌عامل در بالای حافظه قرار دارد و نقش مدیریتی دارد.
- با این روش، CPU همیشه برنامه‌ای برای اجرا دارد و بیکار نمی‌ماند.
- نیازمند مکانیزم‌های مدیریت حافظه و محافظت (Protection) است تا از دسترسی غیرمجاز جلوگیری شود.

Dual-mode Operation

یکی از مفاهیم پایه‌ای و بسیار مهم در سیستم‌های عامل، عملکرد دو حالت (Dual-mode Operation) است. این قابلیت باعث می‌شود که سیستم‌عامل بتواند خودش و منابع حیاتی سیستم را از برنامه‌های کاربر محافظت کند.

توضیح کامل:

🔒 دو حالت اصلی سیستم:

1. User Mode (حالت کاربر):

- زمانی است که یک برنامه کاربر در حال اجراست.
- دسترسی به منابع حیاتی سیستم محدود است.
- نمی‌تواند به طور مستقیم به سخت‌افزار یا دستورهای حساس دسترسی داشته باشد.

2. Kernel Mode (حالت هسته):

- مخصوص کد سیستم‌عامل و برخی عملیات سطح پایین.
- دسترسی کامل به سخت‌افزار و منابع دارد.
- فقط سیستم‌عامل و توابع سطح پایین در این حالت اجرا می‌شوند.

🧠 نقش Mode Bit (بیت وضعیت):

- یک بیت سخت‌افزاری است که تعیین می‌کند سیستم در کدام حالت قرار دارد.
- اگر مقدار این بیت ۰ باشد: سیستم در حالت kernel است.
- اگر مقدار این بیت ۱ باشد: سیستم در حالت user است.

? چطور از سوءاستفاده جلوگیری می‌شود؟

برنامه‌های کاربر اجازه ندارند مستقیماً حالت سیستم را به kernel mode تغییر دهند. تنها راه ورود به حالت kernel:

- است (فراخوانی سیستمی) System Call.

- با system call، سیستم به صورت کنترل شده وارد kernel mode می‌شود.
- پس از انجام عملیات، با دستور return، دوباره به user mode برمی‌گردد.

دستورهای خاص: Privileged Instructions

- برخی دستورهای فقط در حالت kernel قابل اجرا هستند (مثلاً دسترسی مستقیم به I/O یا تغییر حافظه).
- اگر برنامه‌ای در حالت user بخواهد این دستورات را اجرا کند، باعث trap شده و OS کنترل را به دست می‌گیرد.

مثال ساده:

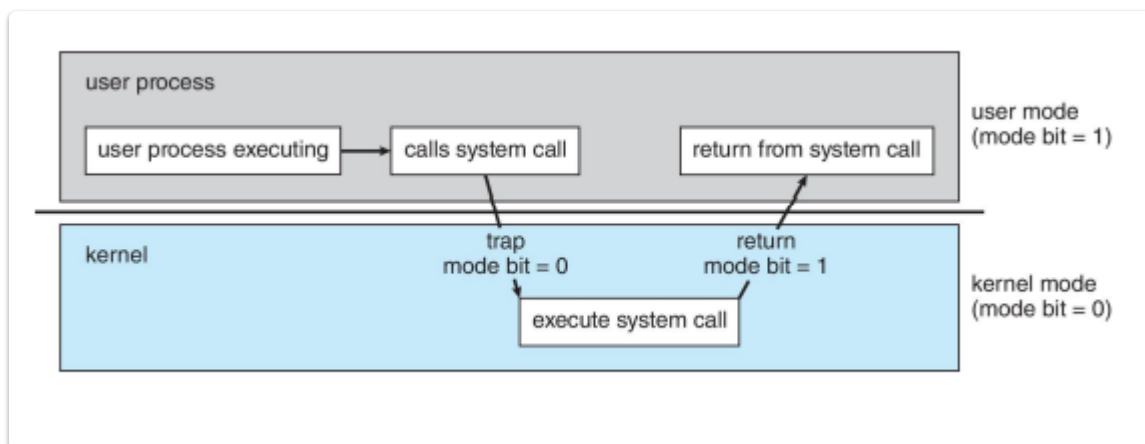
فرض کنید برنامه‌ای بخواهد از فایل بخواند:

1. کاربر در user mode قرار دارد.
2. با فراخوانی system call مثل read()، سیستم وارد kernel mode می‌شود.
3. عملیات خواندن انجام می‌شود.
4. پس از اتمام، سیستم به user mode بازمی‌گردد و نتیجه به برنامه برمی‌گردد.

نکات کلیدی:

- عملکرد دو حالت از دسترسی غیرمجاز و خرابکاری جلوگیری می‌کند.
- تنها سیستم‌عامل می‌تواند در kernel mode اجرا شود.
- انتقال بین حالت‌ها با system call و دستور return انجام می‌شود.
- بیت mode توسط سخت‌افزار کنترل می‌شود، نه کاربر.

تبدیل از حالت کاربر به حالت هسته (User to Kernel Mode)



توضیح نمودار:

ناحیه خاکستری: User Process (فرآیند کاربر)

- برنامه‌ای که در حال اجراست: user process executing

- برنامه نیاز به انجام یک عملیات خاص دارد (مثلاً خواندن از دیسک) و یک فراخوانی سیستمی: **calls system call**: انجام می‌دهد (system call).

● ناحیه آبی: Kernel (هسته سیستم‌عامل)

- **Trap:**
 - نقطه‌ای که سیستم وارد هسته می‌شود.
 - است kernel حالا سیستم در حالت $\rightarrow \text{mode bit} = 0$.
- **execute system call:** (و غیره، `read()`، `write()` مثل) سیستم‌عامل، فراخوانی کاربر را اجرا می‌کند.
- **Return:**
 - پس از انجام عملیات، سیستم از حالت kernel خارج می‌شود.
 - $\rightarrow \text{mode bit} = 1$ user بازگشت به حالت.
- **return from system call:** کنترل به فرآیند کاربر بازمی‌گردد.

🔒 چرا این مکانیزم مهم است؟

- مانع می‌شود که برنامه‌های کاربر مستقیماً به منابع سیستم دسترسی داشته باشند.
- فقط از طریق واسط مشخص (system call) و تحت کنترل سیستم‌عامل این دسترسی ممکن است.
- این ساختار امنیت، پایداری و کنترل بهتر سیستم‌عامل را تضمین می‌کند.

در سیستم‌عامل‌ها Timer

در سیستم‌های عامل، تایمر (Timer) یکی از ابزارهای حیاتی برای مدیریت زمان استفاده از CPU است. هدف اصلی آن، جلوگیری از انحصار منابع توسط یک فرآیند (مانند گیر کردن در حلقه بی‌نهایت یا استفاده طولانی از CPU) می‌باشد.

توضیح کامل:

چرا تایمر لازم است؟

- تصور کنید یک فرآیند وارد حلقه‌ای بی‌پایان شود؛ در این صورت CPU را برای همیشه اشغال می‌کند و سایر فرآیندها هرگز اجرا نمی‌شوند.
- برای جلوگیری از این مشکل، از تایمر سخت‌افزاری استفاده می‌شود.

نحوه کارکرد تایمر:

1. سیستم‌عامل (OS) پیش از واگذاری CPU به یک فرآیند، مقدار مشخصی زمان برای اجرای آن تعیین می‌کند.
 2. این زمان در قالب یک شمارنده (counter) تنظیم می‌شود که با ساعت فیزیکی سیستم (hardware clock) کاهش می‌یابد.
 3. اگر این شمارنده به صفر برسد:
 - وقفه (Interrupt) تولید می‌شود.
 - کنترل سیستم از فرآیند به سیستم‌عامل بازمی‌گردد.
- تصمیم می‌گیرد که OS:

- فرآیند را متوقف یا تعویض کند.
- یا اگر زمان کافی دارد، مجدداً ادامه دهد.

ویژگی مهم:

- تنظیم تایمر تنها توسط سیستم عامل انجام می شود (از طریق دستورالعمل های سطح دسترسی بالا یا privileged instructions)، تا از سوءاستفاده توسط برنامه ها جلوگیری شود.

مثال ساده:

فرض کنید فرآیند A قرار است حداکثر 50 میلی ثانیه از CPU استفاده کند. سیستم عامل مقدار تایمر را روی 50ms تنظیم می کند. اگر قبل از پایان این مدت، فرآیند A به پایان نرسید، تایمر با پایان شمارش خود باعث ایجاد وقفه می شود و کنترل به سیستم عامل بازمی گردد.

نکات کلیدی:

- تایمر برای مدیریت زمان استفاده از CPU و جلوگیری از انحصار منابع استفاده می شود.
- تایمر به کمک ساعت سخت افزاری سیستم کار می کند.
- تولید وقفه زمانی که شمارنده به صفر برسد، باعث بازگشت کنترل به سیستم عامل می شود.
- فقط سیستم عامل اجازه دارد تایمر را تنظیم کند (privileged instruction).

مدیریت فرآیند (Process Management)

تعریف فرآیند

- فرآیند یعنی اجرای یک برنامه. در واقع، فرآیند یک واحد کاری فعال در سیستم است.
- تفاوت برنامه و فرآیند:
- برنامه یک موجودیت غیرفعال (Passive) است، یعنی فقط کد و داده ها به صورت فایل ذخیره شده اند.
- فرآیند یک موجودیت فعال (Active) است که برنامه را در حال اجرا نشان می دهد.

منابع مورد نیاز فرآیند

یک فرآیند برای انجام کار خود به منابع زیر نیاز دارد:

- پردازنده CPU
- حافظه
- دستگاه های ورودی/خروجی (I/O)
- فایل ها
- داده های اولیه (Initialization data)

خاتمه فرآیند

- وقتی فرآیند به پایان می‌رسد، باید منابعی که قابل استفاده مجدد هستند (مثل حافظه و فایل‌ها) آزاد شوند.

شمارنده برنامه (Program Counter)

- در فرآیندهای تک‌ریسمانی (Single-threaded)، یک شمارنده برنامه وجود دارد که آدرس دستور بعدی برای اجرا را مشخص می‌کند.
- فرآیند دستورات را به صورت متوالی و یکی‌یکی اجرا می‌کند تا تمام شود.
- در فرآیندهای چندریسمانی (Multi-threaded)، به ازای هر ریسمان یک شمارنده برنامه وجود دارد.

هم‌زمانی (Concurrency)

- معمولاً در سیستم چندین فرآیند وجود دارد که برخی متعلق به کاربران و برخی سیستم عامل هستند.
- این فرآیندها به صورت هم‌زمان روی یک یا چند CPU اجرا می‌شوند.
- هم‌زمانی از طریق زمان‌بندی و تقسیم زمان CPU بین فرآیندها و ریسمان‌ها انجام می‌شود.

مثال ساده آموزشی

فرض کنید برنامه‌ای داریم که می‌خواهد 10 فایل را باز کرده و بخواند. این برنامه یک فرآیند است که برای انجام کارش به منابع زیر نیاز دارد:

- برای پردازش دستورها CPU
- حافظه برای ذخیره داده‌های فایل‌ها
- فایل‌ها برای باز و خواندن

فرآیند طی اجرای برنامه، به صورت متوالی (اگر تک‌ریسمانی باشد) یا هم‌زمان (اگر چندریسمانی باشد) این کارها را انجام می‌دهد. وقتی تمام فایل‌ها خوانده شدند، فرآیند به پایان می‌رسد و منابع آزاد می‌شوند.

نکات کلیدی

- فرآیند اجرای فعال یک برنامه است، در حالی که برنامه فقط کد است.
- فرآیند به منابع مختلفی مانند CPU، حافظه و دستگاه‌های I/O نیاز دارد.
- فرآیندهای تک‌ریسمانی دارای یک شمارنده برنامه هستند که دستور بعدی را مشخص می‌کند.
- فرآیندهای چندریسمانی دارای چند شمارنده برنامه هستند، هر کدام برای یک ریسمان.
- سیستم عامل معمولاً چندین فرآیند را به طور هم‌زمان با استفاده از زمان‌بندی CPU مدیریت می‌کند.
- آزادسازی منابع در پایان فرآیند اهمیت بالایی دارد.

فعالیت‌های مدیریت فرآیند (Process Management Activities)

1. ایجاد و حذف فرآیندها

- سیستم عامل مسئول **ایجاد (Creation)** فرآیندهای کاربری و سیستم است.
- همچنین فرآیندها باید بتوانند به درستی **حذف (Termination)** شوند و منابع مربوط به آن‌ها آزاد شود.

2. معلق کردن و از سرگیری فرآیندها

- سیستم عامل باید بتواند فرآیندها را **معلق (Suspend)** کند؛ یعنی اجرای آن‌ها را موقتاً متوقف کند،
- و پس از مدتی، اجرای آن‌ها را دوباره **از سر بگیرد (Resume)**.

3. فراهم کردن مکانیزم‌های همگام‌سازی فرآیندها

- فرآیندها ممکن است برای دسترسی به منابع مشترک یا اجرای کارهای هماهنگ شده نیاز به **همگام‌سازی (Synchronization)** داشته باشند.
- سیستم عامل این مکانیزم‌ها را فراهم می‌کند تا از مشکلاتی مانند شرایط رقابتی (Race Condition) جلوگیری شود.

4. فراهم کردن مکانیزم‌های ارتباط بین فرآیندها

- فرآیندها گاهی باید با یکدیگر **ارتباط (Communication)** برقرار کنند و اطلاعات رد و بدل کنند.
- سیستم عامل ابزارهایی مثل صف پیام (Message Queues)، حافظه مشترک (Shared Memory) و ... را برای این منظور ارائه می‌دهد.

5. مدیریت بن‌بست‌ها (Deadlock Handling)

- در شرایطی که چند فرآیند منتظر منابعی هستند که توسط دیگری قفل شده، بن‌بست (Deadlock) رخ می‌دهد.
- سیستم عامل باید مکانیزم‌هایی برای **پیشگیری، تشخیص و رفع بن‌بست** ارائه دهد.

مثال ساده آموزشی

فرض کنید یک برنامه متن‌باز که ویرایشگر متن است، چند فرآیند دارد:

- یک فرآیند برای نمایش رابط کاربری
- یک فرآیند برای ذخیره فایل به صورت خودکار
- یک فرآیند برای بررسی املای کلمات

سیستم عامل:

- این فرآیندها را ایجاد و حذف می‌کند،
- ممکن است فرآیند ذخیره‌سازی خودکار را هنگام کمبود منابع موقتاً معلق کند،
- مکانیزم‌های همگام‌سازی را برای جلوگیری از تداخل ذخیره‌سازی و ویرایش فراهم می‌کند،
- و ارتباط بین این فرآیندها را برقرار می‌کند.

نکات کلیدی

- مدیریت فرآیند شامل ایجاد، حذف، معلق کردن و از سرگیری فرآیندها است.
- همگام‌سازی و ارتباط بین فرآیندها برای هماهنگی اجرای صحیح ضروری‌اند.
- سیستم عامل باید مکانیزم‌های مدیریت بن‌بست را فراهم کند تا از توقف دائمی سیستم جلوگیری شود.
- این فعالیت‌ها تضمین می‌کنند که سیستم عامل به صورت کارآمد و پایدار فرآیندها را مدیریت کند.

مدیریت حافظه (Memory Management)

اهمیت حافظه در اجرای برنامه‌ها

- برای اجرای یک برنامه، همه یا بخشی از دستورات آن باید در **حافظه اصلی (RAM)** قرار داشته باشند.
- همچنین، همه یا بخشی از داده‌هایی که برنامه به آن‌ها نیاز دارد باید در حافظه باشند تا قابل دسترسی باشند.

هدف مدیریت حافظه

- مدیریت حافظه** مسئول تعیین این است که کدام بخش از برنامه‌ها و داده‌ها در حافظه باشند و چه زمانی باید جابجا شوند.
- هدف اصلی بهینه‌سازی استفاده از CPU و بهبود پاسخگویی سیستم به کاربران است.

فعالیت‌های مدیریت حافظه

- ردیابی حافظه**
 - نگهداری اطلاعات درباره اینکه کدام بخش‌های حافظه در حال حاضر توسط کدام فرآیند یا داده اشغال شده‌اند.
- جابجایی داده‌ها**
 - تصمیم‌گیری درباره اینکه کدام فرآیندها یا بخش‌های آن‌ها و داده‌ها باید به حافظه وارد (load) یا از آن خارج (swap out) شوند.
- اختصاص و آزادسازی حافظه**
 - تخصیص فضای حافظه به فرآیندها هنگام نیاز و آزادسازی آن‌ها پس از اتمام کار.

مثال ساده آموزشی

فرض کنید سه برنامه می‌خواهند روی یک کامپیوتر اجرا شوند، اما حافظه کافی برای همه برنامه‌ها به طور همزمان وجود ندارد.

مدیریت حافظه تصمیم می‌گیرد:

- کدام برنامه‌ها یا بخش‌های آن‌ها باید در حافظه باشند و اجرا شوند،
- کدام برنامه‌ها را موقتاً به دیسک منتقل کند (swap out)،
- و زمانی که برنامه‌ای کارش تمام شد، فضای حافظه را آزاد کند تا برنامه دیگری بتواند از آن استفاده کند.

نکات کلیدی

- اجرای برنامه نیازمند قرار گرفتن دستورات و داده‌های آن در حافظه است.
- مدیریت حافظه مسئول بهینه‌سازی استفاده از حافظه و افزایش کارایی سیستم است.
- فعالیت‌های اصلی شامل ردیابی، جابجایی، تخصیص و آزادسازی حافظه است.
- تصمیمات مدیریت حافظه تأثیر مستقیم روی سرعت اجرای برنامه‌ها و پاسخگویی سیستم دارند.

مدیریت سیستم فایل (File-System Management)

دیدگاه کلی سیستم عامل نسبت به ذخیره‌سازی اطلاعات

- سیستم عامل یک **نمایش منطقی و یکپارچه** از فضای ذخیره‌سازی اطلاعات ارائه می‌دهد.
- این نمایش منطقی به نام **فایل (File)** است که ویژگی‌های فیزیکی دستگاه‌های ذخیره‌سازی (مثل دیسک، نوار مغناطیسی) را از کاربر پنهان می‌کند.

ویژگی‌های دستگاه‌های ذخیره‌سازی

- هر دستگاه توسط یک کنترلر مخصوص مدیریت می‌شود (مثل کنترلر دیسک).
- دستگاه‌ها در خصوصاتی مثل سرعت دسترسی، ظرفیت، نرخ انتقال داده و روش دسترسی (دستیابی ترتیبی یا تصادفی) متفاوت هستند.

مدیریت سیستم فایل

- فایل‌ها معمولاً در قالب ساختارهایی به نام **دایرکتوری‌ها (Directories)** سازماندهی می‌شوند.
- سیستم عامل دسترسی به فایل‌ها را کنترل می‌کند و تعیین می‌کند چه کسی به کدام فایل‌ها اجازه دسترسی دارد (کنترل دسترسی).

وظایف سیستم عامل در مدیریت فایل‌ها

- **ایجاد و حذف** فایل‌ها و دایرکتوری‌ها

- ارائه عملیات اولیه (Primitives) برای **دستکاری فایل‌ها و دایرکتوری‌ها** (مثل باز کردن، خواندن، نوشتن، بستن)
- **نگاشت فایل‌ها به حافظه ثانویه** (دیسک یا سایر رسانه‌ها)
- **پشتیبان‌گیری فایل‌ها** روی رسانه‌های پایدار و غیر فزّار (مثل دیسک سخت یا نوار مغناطیسی)

مثال ساده آموزشی

فرض کنید یک کاربر می‌خواهد یک فایل متنی بسازد و در آن متنی ذخیره کند:

- سیستم عامل ابتدا یک فایل جدید در دایرکتوری مشخص ایجاد می‌کند.
- سپس کاربر می‌تواند با عملیات خواندن و نوشتن، داده‌ها را در فایل وارد کند.
- سیستم عامل داده‌ها را روی دیسک سخت ذخیره می‌کند و دسترسی به این فایل را برای کاربر و دیگران کنترل می‌کند.
- در صورت نیاز، سیستم عامل فایل‌ها را روی رسانه‌های پشتیبان ذخیره می‌کند تا در صورت خرابی دیسک، داده‌ها از دست نروند.

نکات کلیدی

- سیستم فایل یک نمای منطقی و انتزاعی از فضای ذخیره‌سازی است که دستگاه‌های مختلف را یکسان می‌کند.
- فایل‌ها به صورت سازمان‌یافته در دایرکتوری‌ها قرار می‌گیرند.
- سیستم عامل مسئول کنترل دسترسی، نگاشت فایل‌ها و عملیات روی آن‌ها است.
- پشتیبان‌گیری از داده‌ها برای حفظ امنیت و قابلیت بازیابی اطلاعات ضروری است.

مدیریت ذخیره‌سازی حجیم (Mass-Storage Management)

اهمیت ذخیره‌سازی حجیم

- دیسک‌ها معمولاً برای ذخیره داده‌هایی استفاده می‌شوند که در حافظه اصلی جا نمی‌شوند یا باید برای مدت زمان طولانی نگهداری شوند.
- مدیریت صحیح این ذخیره‌سازی اهمیت زیادی دارد چون عملکرد کل سیستم کامپیوتری به عملکرد زیرسیستم دیسک وابسته است.

نقش سیستم عامل در مدیریت ذخیره‌سازی حجیم

سیستم عامل فعالیت‌های مهمی را برای مدیریت ذخیره‌سازی انجام می‌دهد، از جمله:

1. سوارکردن و جداکردن دستگاه‌های ذخیره‌سازی (Mounting & Unmounting)

- سوارکردن یعنی اتصال دستگاه ذخیره‌سازی به سیستم فایل برای دسترسی به داده‌ها.
- جداکردن یعنی قطع این اتصال به صورت ایمن.

2. مدیریت فضای آزاد (Free-space Management)

- پیگیری بخش‌های خالی حافظه ذخیره‌سازی برای تخصیص به فایل‌ها و داده‌های جدید.

3. تخصیص فضای ذخیره‌سازی (Storage Allocation)

- تصمیم‌گیری درباره اینکه داده‌ها در کدام بخش‌های دیسک ذخیره شوند.

4. زمان‌بندی دیسک (Disk Scheduling)

- ترتیب‌بندی دسترسی به درخواست‌های مختلف خواندن/نوشتن برای بهینه‌سازی عملکرد دیسک.

5. تقسیم‌بندی (Partitioning)

- تقسیم دیسک به بخش‌های منطقی مختلف که هر کدام می‌توانند سیستم فایل یا کاربرد خاصی داشته باشند.

6. حفاظت (Protection)

- کنترل دسترسی و محافظت داده‌ها در برابر دسترسی غیرمجاز.

مثال ساده آموزشی

فرض کنید دیسک سخت یک کامپیوتر 1 ترابایت ظرفیت دارد و چندین کاربر فایل‌هایی روی آن ذخیره می‌کنند.

- سیستم عامل مسئول مدیریت فضای آزاد برای جلوگیری از پر شدن دیسک است.
- همچنین، برای افزایش سرعت، درخواست‌های خواندن و نوشتن روی دیسک زمان‌بندی می‌شوند.
- وقتی یک دستگاه جدید وصل می‌شود، سیستم عامل آن را سوار می‌کند تا قابل استفاده باشد.
- دیسک به بخش‌های منطقی تقسیم شده تا سیستم عامل بتواند چند سیستم فایل را به صورت مستقل مدیریت کند.

نکات کلیدی

- دیسک‌ها برای ذخیره داده‌های بزرگ و بلندمدت استفاده می‌شوند.
- مدیریت ذخیره‌سازی حجیم تاثیر زیادی روی سرعت و عملکرد کلی سیستم دارد.
- سیستم عامل باید عملیات متنوعی مانند سوارکردن، تخصیص فضا، زمان‌بندی و حفاظت را انجام دهد.
- مدیریت صحیح این فعالیت‌ها باعث افزایش کارایی و امنیت داده‌ها می‌شود.

کشینگ (Caching)

مفهوم کشینگ

- کشینگ یک اصل مهم در کامپیوتر است که در سطوح مختلف انجام می‌شود:
 - سخت‌افزار (مثل کش CPU)
 - سیستم عامل
 - نرم‌افزارهای کاربردی
- هدف کشینگ این است که اطلاعاتی که در حال استفاده هستند، از حافظه یا ذخیره‌سازی کندتر به حافظه سریع‌تر به صورت موقت کپی شوند.

نحوه کار کشینگ

- سیستم ابتدا **حافظه سریع تر (کش)** را بررسی می کند تا ببیند اطلاعات مورد نظر آنجا هست یا خیر.
- اگر اطلاعات در کش موجود باشد (حالت **Cache Hit**)، مستقیماً از کش استفاده می شود که بسیار سریع تر است.
- اگر اطلاعات در کش نباشد (حالت **Cache Miss**)، داده ها از حافظه کندتر خوانده شده و در کش کپی می شوند، سپس استفاده می شوند.

ویژگی های کش

- اندازه کش معمولاً **کوچکتر از حافظه یا ذخیره سازی اصلی** است که اطلاعات از آن کپی می شود.
- مدیریت کش** از مسائل مهم طراحی است که شامل:
 - اندازه کش
 - سیاست جایگزینی (Replacement Policy) برای انتخاب داده ای که هنگام پر شدن کش حذف می شود.

مثال ساده آموزشی

فرض کنید CPU می خواهد داده ای را از حافظه بخواند:

- ابتدا کش CPU را بررسی می کند. اگر داده آنجا باشد (مثلاً مقدار متغیری که اخیراً استفاده شده)، خیلی سریع آن را می خواند.
- اگر داده در کش نباشد، حافظه اصلی را می خواند و آن داده را در کش ذخیره می کند تا دفعه بعد سریع تر در دسترس باشد.

نکات کلیدی

- کشینگ باعث افزایش سرعت دسترسی به داده ها می شود با استفاده از حافظه سریع تر موقت.
- اندازه و سیاست جایگزینی کش نقش مهمی در عملکرد کلی سیستم دارند.
- کش در سطوح مختلف سیستم (سخت افزار، سیستم عامل، نرم افزار) استفاده می شود.
- مدیریت بهینه کش یک چالش طراحی کلیدی است.

مهاجرت داده "A" از دیسک به رجیستر (Migration of Data "A" from Disk to Register)

اهمیت استفاده از جدیدترین مقدار داده در سیستم های چندوظیفه ای

- در محیط های **چندوظیفه ای (Multitasking)** باید مطمئن شویم که همواره **جدیدترین مقدار** داده استفاده می شود، بدون توجه به اینکه داده در کدام لایه از سلسله مراتب حافظه (دیسک، کش، حافظه اصلی، رجیستر) قرار دارد.

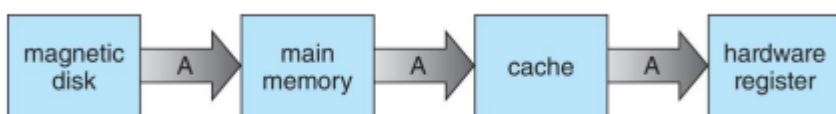
هماهنگی کش در محیط‌های چندپردازنده‌ای (Multiprocessor)

- در محیط‌های چندپردازنده‌ای، هر CPU ممکن است نسخه‌ای از داده را در **کش خود** داشته باشد.
- سیستم باید **هماهنگی کش (Cache Coherency)** را به صورت سخت‌افزاری فراهم کند، به طوری که همه CPUها همیشه جدیدترین مقدار داده را در کش خود داشته باشند.

پیچیدگی بیشتر در محیط‌های توزیع‌شده (Distributed Environment)

- در سیستم‌های توزیع‌شده، ممکن است چند نسخه از یک داده در مکان‌های مختلف وجود داشته باشد.
- حفظ سازگاری این نسخه‌ها چالش‌برانگیز است و راه‌حل‌های مختلفی برای آن وجود دارد (که در فصل 19 کتاب استالینگز بررسی شده‌اند).

دیاگرام – مسیر مهاجرت داده از دیسک به رجیستر و چالش‌های هماهنگی



نکات کلیدی

- در سیستم‌های چندوظیفه‌ای و چندپردازنده‌ای، استفاده از جدیدترین مقدار داده حیاتی است.
- هماهنگی کش سخت‌افزاری برای حفظ سازگاری داده بین CPUها ضروری است.
- در محیط‌های توزیع‌شده، چالش‌های بیشتری برای مدیریت چند نسخه داده وجود دارد.
- راهکارهای متنوعی برای حل این مشکلات وجود دارد که در فصل‌های پیشرفته‌تر بررسی می‌شوند.

زیرسیستم ورودی/خروجی (I/O Subsystem)

هدف کلی سیستم عامل در مدیریت دستگاه‌های سخت‌افزاری

- یکی از اهداف مهم سیستم عامل این است که **ویژگی‌ها و پیچیدگی‌های خاص دستگاه‌های سخت‌افزاری را از دید کاربر پنهان کند**.
- کاربر نیازی ندارد نحوه کار دقیق دستگاه‌ها را بداند، بلکه فقط باید بتواند به سادگی از آن‌ها استفاده کند.

وظایف زیرسیستم ورودی/خروجی

1. مدیریت حافظه مرتبط با I/O

- شامل **بافرینگ (Buffering)**: ذخیره موقت داده‌ها هنگام انتقال بین دستگاه‌ها یا بین دستگاه و حافظه،
- **کشینگ (Caching)**: ذخیره بخش‌هایی از داده در حافظه سریع‌تر برای افزایش کارایی،
- **اسپولینگ (Spooling)**: اجرای هم‌زمان عملیات خروجی یک کار با ورودی کارهای دیگر، برای بهبود بهره‌وری سیستم.

2. رابط کلی درایور دستگاه (Device-Driver Interface)

- فراهم کردن یک رابط استاندارد برای ارتباط سیستم عامل با انواع مختلف درایورها.

3. درایورهای مخصوص دستگاه‌ها

- هر سخت‌افزار خاص، نیازمند یک درایور مخصوص است که وظیفه کنترل و مدیریت آن دستگاه را برعهده دارد.

مثال ساده آموزشی

فرض کنید کاربر می‌خواهد فایلی را پرینت کند:

- داده‌های فایل ابتدا در حافظه بافر ذخیره می‌شوند (buffering) تا پرینتر بتواند آن‌ها را دریافت کند.
- اگر بخشی از داده‌ها اخیراً پرینت شده و دوباره نیاز باشد، ممکن است در کش ذخیره شده باشند تا سریع‌تر پردازش شوند.
- در این حین، اسپولینگ اجازه می‌دهد پرینتر خروجی یک فایل را چاپ کند و هم‌زمان کاربر فایل دیگری را ارسال کند.
- درایور پرینتر وظیفه دارد که دستورات مناسب را به پرینتر ارسال و پاسخ آن را دریافت کند.

نکات کلیدی

- زیرسیستم I/O پیچیدگی‌های سخت‌افزاری را از کاربر مخفی می‌کند.
- مدیریت حافظه برای I/O شامل بافرینگ، کشینگ و اسپولینگ است.
- رابط درایور دستگاه، ارتباط بین سیستم عامل و درایورها را تسهیل می‌کند.
- هر دستگاه سخت‌افزاری درایور مخصوص به خود را دارد که توسط سیستم عامل استفاده می‌شود.

حفاظت و امنیت (Protection and Security)

تفاوت حفاظت و امنیت

- **حفاظت (Protection)**: مجموعه مکانیزم‌هایی است که دسترسی کاربران یا فرآیندها به منابع سیستم را کنترل می‌کند. هدف اصلی آن جلوگیری از دسترسی غیرمجاز به منابع است.
- **امنیت (Security)**: حفاظت سیستم در برابر حملات داخلی و خارجی است، مانند:
 - حملات انکار سرویس (DoS)
 - ویروس‌ها و کرم‌ها
 - سرقت هویت
 - سرقت خدمات

مدیریت دسترسی کاربران

- سیستم‌ها معمولاً ابتدا کاربران را از هم تمیز می‌دهند تا تعیین کنند هر کاربر چه مجوزهایی دارد.

شناسه کاربری (User ID)

- هر کاربر دارای یک شناسه یکتا است که شامل نام و عددی مخصوص است.
- این شناسه به فایل‌ها، فرآیندها و منابع دیگر مرتبط می‌شود تا کنترل دسترسی اعمال شود.

شناسه گروهی (Group ID)

- کاربران می‌توانند در گروه‌ها دسته‌بندی شوند.
- این شناسه گروه، مجموعه‌ای از کاربران را تعریف کرده و کنترل دسترسی را ساده‌تر می‌کند.
- هر فایل یا فرآیند هم می‌تواند به یک گروه مرتبط باشد.

افزایش سطح دسترسی (Privilege Escalation)

- گاهی یک کاربر می‌تواند به شناسه کاربری دیگری (معمولاً با حقوق بالاتر) تغییر کند تا عملیات خاصی را انجام دهد.
- این مکانیزم باید با دقت مدیریت شود تا امنیت سیستم حفظ شود.

مثال ساده آموزشی

فرض کنید کاربری با شناسه کاربری `User123` در یک سیستم کار می‌کند:

- این شناسه مشخص می‌کند که کاربر می‌تواند به کدام فایل‌ها دسترسی داشته باشد.
- اگر کاربر عضو گروه `Admin` باشد، فایل‌هایی که به این گروه تعلق دارند نیز قابل دسترسی خواهند بود.
- برای اجرای عملیاتی که نیاز به سطح دسترسی بالاتر دارد، کاربر ممکن است از طریق افزایش سطح دسترسی به شناسه‌ای با حقوق بیشتر (مثل `root`) تغییر کند.

نکات کلیدی

- حفاظت کنترل دسترسی به منابع را برعهده دارد.
- امنیت شامل مقابله با تهدیدات و حملات خارجی و داخلی است.
- شناسه‌های کاربری و گروهی، پایه کنترل دسترسی در سیستم‌ها هستند.
- افزایش سطح دسترسی باید با دقت و امنیت مدیریت شود تا سوءاستفاده جلوگیری شود.

مجازی‌سازی (Virtualization)

مفهوم مجازی‌سازی

- مجازی‌سازی** به سیستم‌عامل‌ها اجازه می‌دهد که برنامه‌ها را در داخل سیستم‌عامل‌های دیگر اجرا کنند.

- این فناوری صنعتی بزرگ و رو به رشد است که کاربردهای فراوانی در سرورها، رایانش ابری، توسعه نرم افزار و تست محیط های مختلف دارد.
-

روش های اجرای برنامه ها در مجازی سازی

1. شبیه سازی (Emulation)

- وقتی نوع CPU مبدا با نوع CPU هدف متفاوت باشد (مثلاً اجرای برنامه ای نوشته شده برای PowerPC روی Intel x86)، از شبیه سازی استفاده می شود.
- این روش معمولاً کندترین روش است، چون تمام دستورات باید ترجمه و شبیه سازی شوند.

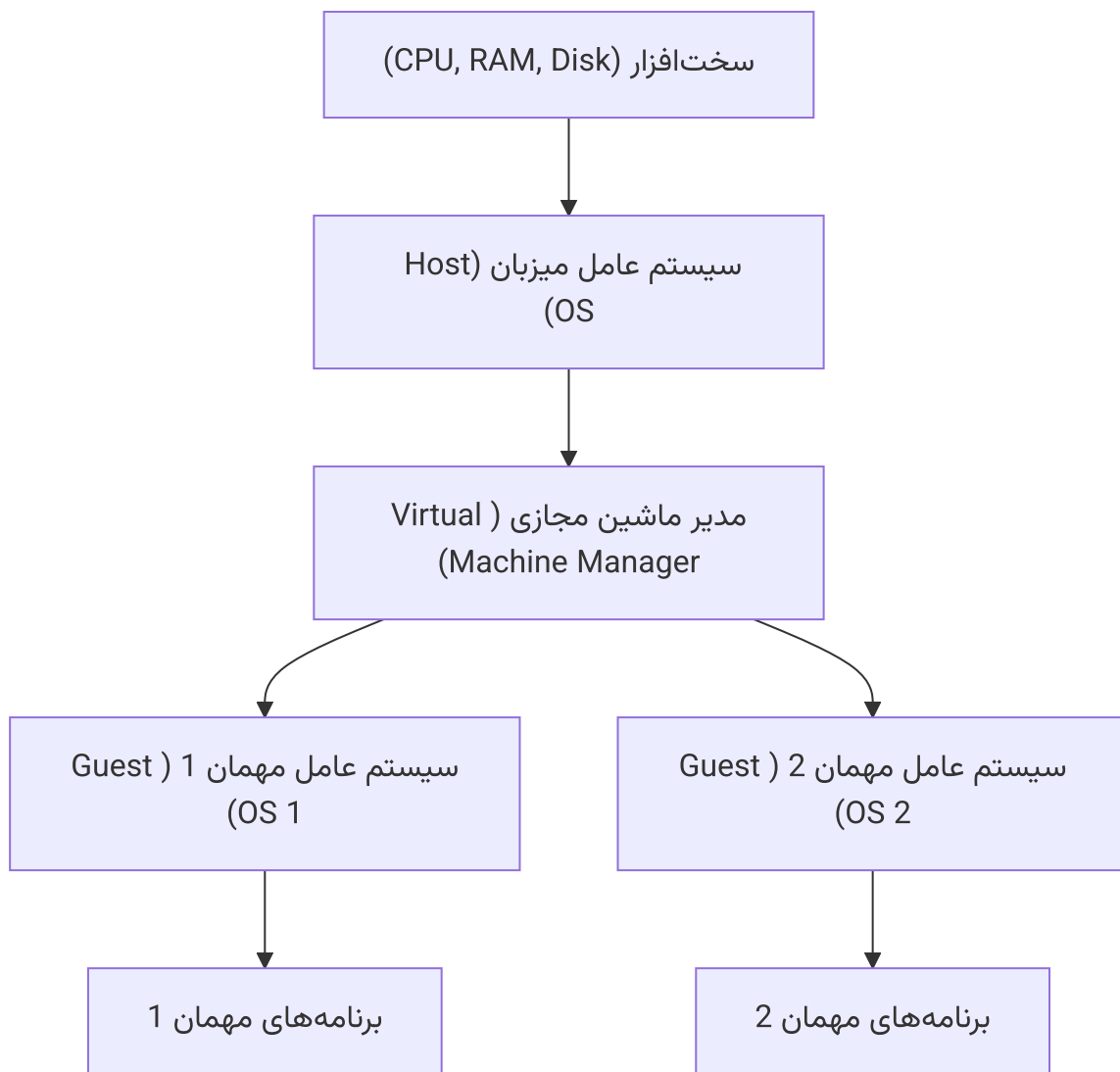
2. تفسیر (Interpretation)

- زمانی استفاده می شود که زبان برنامه نویسی به کد بومی (native code) ترجمه نشده باشد.
- کد به صورت خط به خط خوانده و اجرا می شود که سرعت پایین تری دارد.

3. مجازی سازی (Virtualization)

- سیستم عامل اصلی (host OS) و سیستم های عامل مهمان (guest OSes) هر دو به صورت بومی (نیتیو) برای CPU کامپایل شده اند.
 - مجازی ساز یا مدیر ماشین مجازی (Virtual Machine Manager - VMM) خدمات لازم برای اجرای چندین سیستم عامل مهمان را فراهم می کند.
 - مثال: VMware که سیستم عامل ویندوز XP را به عنوان سیستم مهمان روی میزبان ویندوز XP اجرا می کند، هر دو به صورت نیتیو روی یک سخت افزار.
-

دیاگرام Mermaid – ساختار مجازی سازی



مثال ساده آموزشی

فرض کنید روی یک کامپیوتر با ویندوز 10 (Host OS) از VMware استفاده می کنید:

- شما می توانید یک سیستم عامل مهمان مانند لینوکس را روی ویندوز اجرا کنید.
- هر سیستم عامل مهمان به صورت مستقل برنامه های خودش را اجرا می کند، اما همه روی یک سخت افزار مشترک به کمک مجازی ساز کار می کنند.

نکات کلیدی

- مجازی سازی به سیستم عامل ها اجازه می دهد برنامه ها را در محیط های مختلف بدون تغییر اجرا کنند.
- شبیه سازی برای CPU های متفاوت است و کندتر است.
- مجازی سازها امکان اجرای همزمان چند سیستم عامل مهمان را روی یک میزبان فراهم می کنند.
- نقش اصلی در مدیریت منابع و اجرای مهمان ها دارد VMM.

مجازی سازی (ادامه)

موارد کاربرد مجازی‌سازی

- اجرای چند سیستم‌عامل روی لپ‌تاپ‌ها و دسکتاپ‌ها برای تست، سازگاری و یادگیری:
- مثال: لپ‌تاپ اپل با macOS به عنوان میزبان و ویندوز به عنوان سیستم‌عامل مهمان.
- توسعه نرم‌افزار برای چندین سیستم‌عامل بدون نیاز به داشتن چند دستگاه فیزیکی مختلف.
- آزمایش تضمین کیفیت (QA) برنامه‌ها روی سیستم‌عامل‌ها و محیط‌های متفاوت، بدون نیاز به چند سیستم فیزیکی.
- اجرای و مدیریت محیط‌های پردازی در دیتا سنترها برای بهره‌وری بهتر و مقیاس‌پذیری.

مجازی‌سازهای نوع Native (Bare-metal)

- برخی مدیران ماشین مجازی (VMM) مستقیماً روی سخت‌افزار اجرا می‌شوند و خود به عنوان میزبان عمل می‌کنند.
- در این حالت، سیستم‌عاملی به عنوان میزبان وجود ندارد (یا کمتر دیده می‌شود).
- نمونه‌ها:
- VMware ESX
- Citrix XenServer

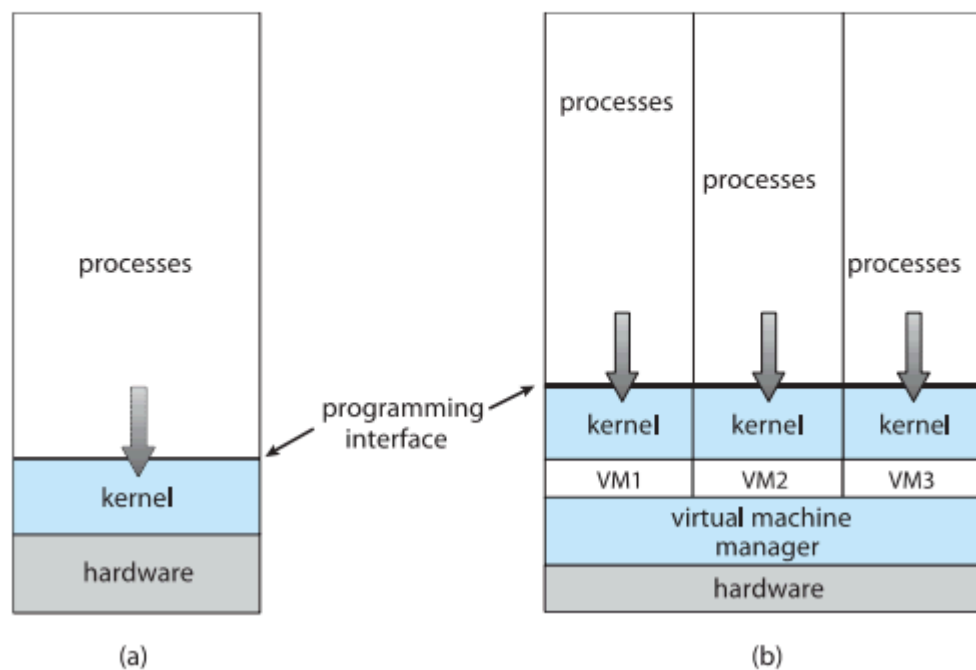
مثال ساده آموزشی

- روی لپ‌تاپ شخصی با ویندوز، شما VMware Workstation را نصب می‌کنید (نوع Hosted):
- سیستم‌عامل ویندوز میزبان است، VMware به عنوان برنامه‌ای روی آن اجرا می‌شود و لینوکس را به عنوان سیستم‌عامل مهمان اجرا می‌کند.
- در دیتاسنتر، سروری با VMware ESX (نوع Bare-metal) وجود دارد:
- این نرم‌افزار مستقیماً روی سخت‌افزار نصب شده و چندین سیستم‌عامل مهمان را بدون واسطه اجرا می‌کند.

نکات کلیدی

- مجازی‌سازی کاربردهای گسترده‌ای در توسعه، تست و مدیریت سیستم‌ها دارد.
 - نوع Hosted روی سیستم‌عامل میزبان اجرا می‌شود، اما نوع Bare-metal مستقیماً روی سخت‌افزار.
 - مدیران ماشین مجازی Bare-metal معمولاً عملکرد بهتری دارند و برای محیط‌های حرفه‌ای استفاده می‌شوند.
 - هستند نمونه‌های معروف مجازی‌ساز VMware ESX و Citrix XenServer.
-

Computing Environments - Virtualization



سیستم‌های توزیع‌شده (Distributed Systems)

تعریف سیستم توزیع‌شده

- مجموعه‌ای از سیستم‌های جداگانه و احتمالاً ناهمگون (متفاوت از نظر سخت‌افزار، سیستم عامل و...) که از طریق شبکه به هم متصل شده‌اند.
- شبکه مسیر ارتباطی بین این سیستم‌ها را فراهم می‌کند.

انواع شبکه‌ها در سیستم‌های توزیع‌شده

- شبکه‌های محلی (LAN - Local Area Network): شبکه‌های کوچک در یک محدوده جغرافیایی محدود، مانند یک ساختمان یا دفتر.
- شبکه‌های گسترده (WAN - Wide Area Network): شبکه‌های بزرگ که مناطق جغرافیایی وسیعی را پوشش می‌دهند، مانند اینترنت.
- شبکه‌های شهری (MAN - Metropolitan Area Network): شبکه‌ای در محدوده یک شهر یا منطقه بزرگ‌تر از LAN ولی کوچکتر از WAN.
- شبکه‌های شخصی (PAN - Personal Area Network): شبکه‌های کوچک برای ارتباط بین دستگاه‌های شخصی نزدیک به هم، مثل بلوتوث.

سیستم عامل شبکه (Network Operating System)

- این سیستم عامل قابلیت هایی را فراهم می کند تا سیستم های مختلف در شبکه بتوانند به هم متصل شده و با یکدیگر پیام رد و بدل کنند.
 - هدف این است که به کاربران و برنامه ها **توهم یک سیستم یکپارچه و واحد** داده شود، در حالی که سیستم ها مستقل و جداگانه هستند.
-

مثال ساده آموزشی

فرض کنید چند کامپیوتر در یک دفتر کاری به شبکه LAN متصل هستند:

- هر کامپیوتر سیستم عاملی مستقل دارد، اما سیستم عامل شبکه اجازه می دهد که این کامپیوترها فایل ها را به اشتراک بگذارند و پیام هایی بین هم ارسال کنند.
 - به این ترتیب، کاربران احساس می کنند که در یک سیستم واحد کار می کنند، در حالی که هر کامپیوتر مجزا است.
-

نکات کلیدی

- سیستم های توزیع شده شامل چند سیستم جداگانه متصل به شبکه هستند.
 - انواع شبکه ها شامل LAN، WAN، MAN و PAN هستند.
 - سیستم عامل شبکه امکان ارتباط و همکاری بین سیستم ها را فراهم می کند.
 - هدف ایجاد توهم یک سیستم واحد برای کاربران و برنامه ها است.
-

End of Chapter 1 :)